

Data Mining and Big data anlysis

数据挖掘及大数据分析

Chpter 5：数据仓库与联机分析处理

02-数据仓库

王东、张谦、丁玥

上海交通大学软件学院RFID与物联网实验室

2025年春季

数据服务与数据分析

- **数据服务与OLTP**
  - **数据服务**
    - ✓ 主要是指面向各种操作型业务，提供数据的增加、删除、修改、简单查询功能的服务
    - ✓ 提供数据服务功能的系统就是数据服务系统，包括支撑操作型业务的关系数据库、NoSQL数据库以及NewSQL数据库
  - **OLTP（on-line transaction processing，联机事务处理）**
    - ✓ OLTP主要用来记录某类业务事件的发生，如购买行为，当行为产生后，系统会记录是谁在何时何地做了何事，这样的一行（或多行）数据会以增删改的方式在数据库中进行数据的更新处理操作，要求实时性高、稳定性强、确保数据及时更新成功，常见的企业业务系统如ERP、CRM、OA等系统都属于OLTP
    - ✓ OLTP要求的响应时间一般是几秒钟时间，比如在银行的存款、取款、查账、转账等业务中

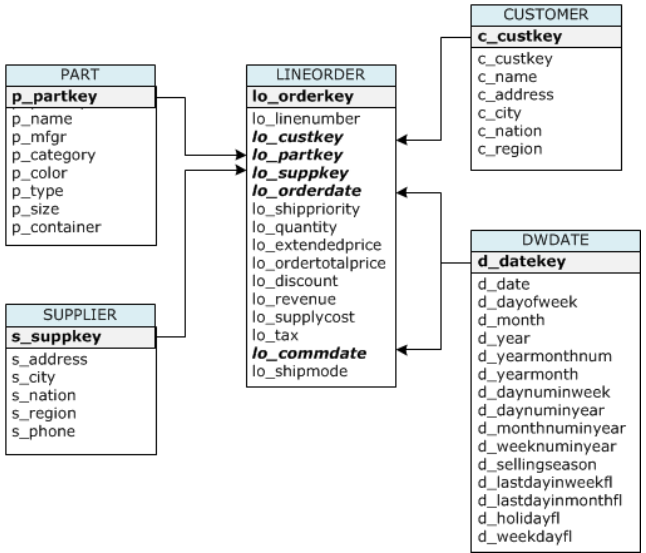
数据服务与数据分析

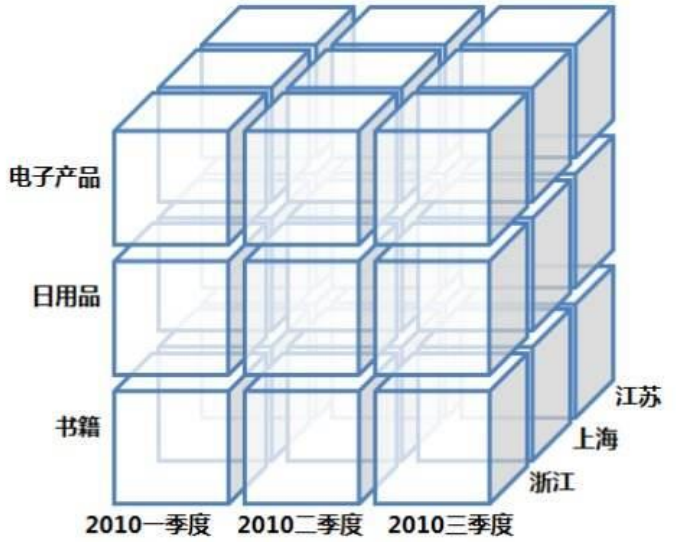
- **数据分析、简单分析、复杂分析**
  - 数据分析
    - ✓ 大量的业务系统采用数据库来进行数据管理以后，随着业务的不断发展，积攒了大量的业务数据。**数据分析任务是面向分析人员（或者管理层），通过对大量数据（甚至在整个数据库数据上）使用复杂的计算模型进行分析，从而发现数据中隐藏的模式或规律**，侧重对决策人员和高层管理人员的决策支持，可以根据分析人员的要求快速、灵活地进行大数据量的复杂查询处理，并且以一种直观而易懂的形式将查询结果提供给决策人员，以便准确掌握企业（公司）的经营状况并进行宏观决策。**这类任务的响应时间一般比较长，比如以分钟和小时计**
    - ✓ **为了避免这类数据处理任务对业务顺利运行产生干扰，一般在业务数据库之外建立数据仓库系统。它从业务数据库抽取、转换和装载(Extract Transform and Load, 简称ETL)数据，帮助人们在上面执行各种分析任务**



数据服务与数据分析

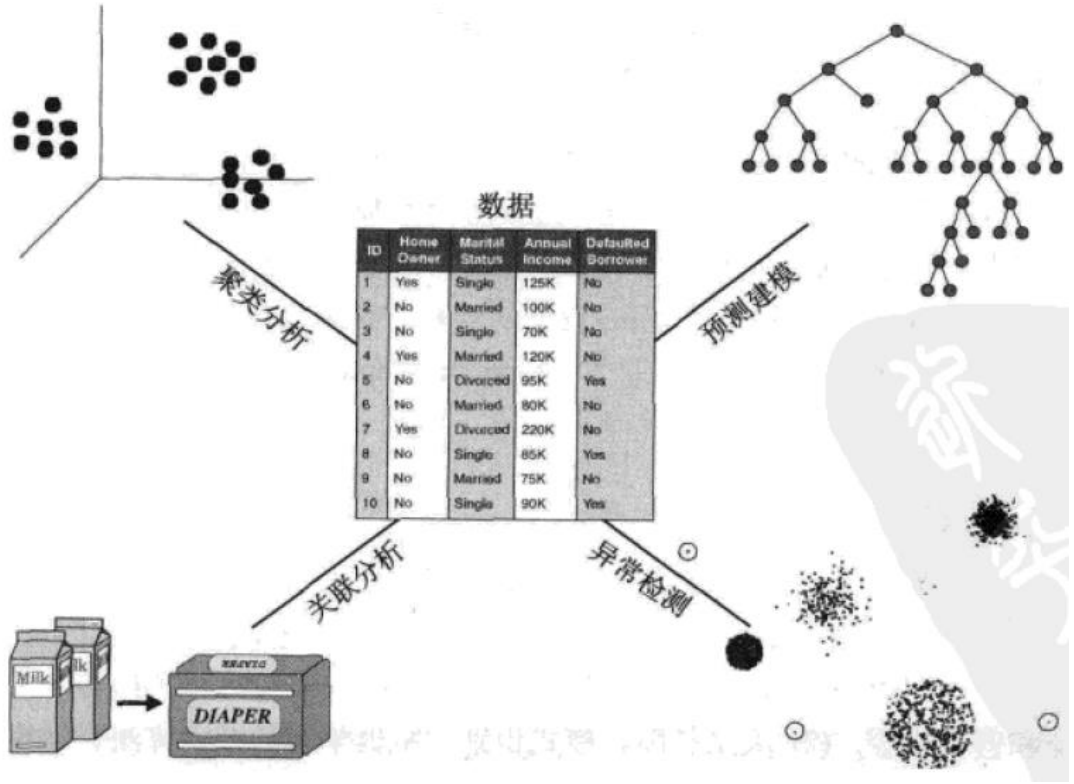
- 数据分析、简单分析、复杂分析
  - 简单分析与OLAP（Online Analytic Processing，联机分析处理）
    - ✓ 简单分析是简单汇总和报表。OLAP采用星型模型或者数据立方体组织数据，保存在数据仓库中，然后进行多维度的聚集查询处理
    - ✓ 在OLAP应用中，在事实表中记录一系列的事件，在维表中记录事件的维度，然后从维表中选取一些维度对事实表进行汇总操作，这些汇总操作包括计数、求和、求平均值、求最大值、求最小值
    - ✓ 将OLAP分析所用到的多维数据，物理上存储为多维数组形式，形成“立方体”(Cube)的结构。星型模型的各个维的属性值，被映射成多维数组的下标值或下标的范围，而聚集数据作为多维数组的值存储在数组的单元中





数据服务与数据分析

- 数据分析、简单分析、**复杂分析**
  - **复杂分析与数据挖掘、机器学习**
    - ✓ 复杂分析一般需要利用统计分析方法、数据挖掘与机器学习方法，**对数据进行深入分析**，提取有用信息并形成结论，辅助决策

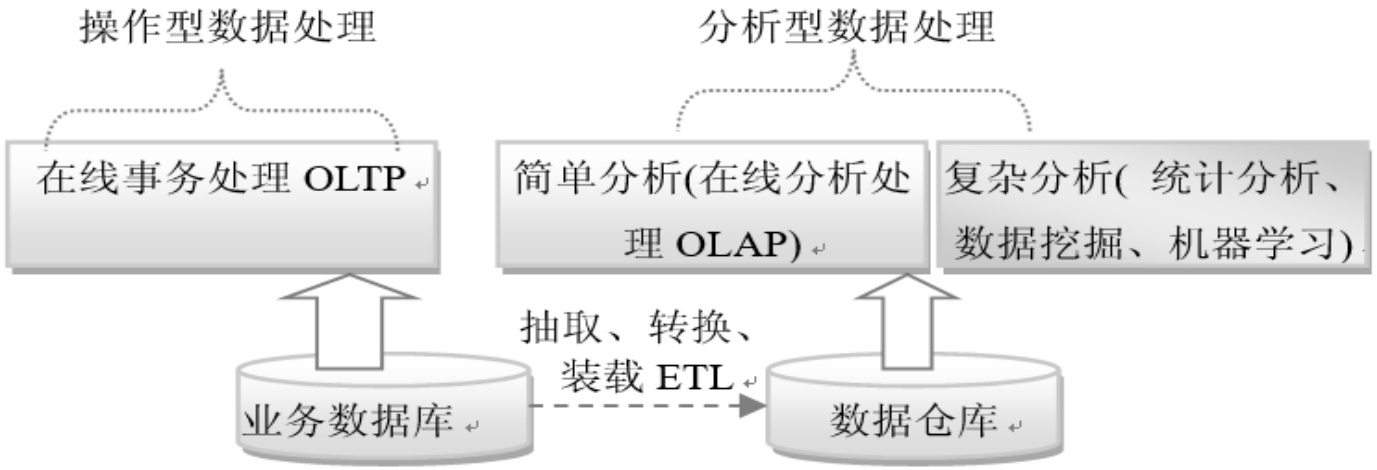


数据服务与数据分析

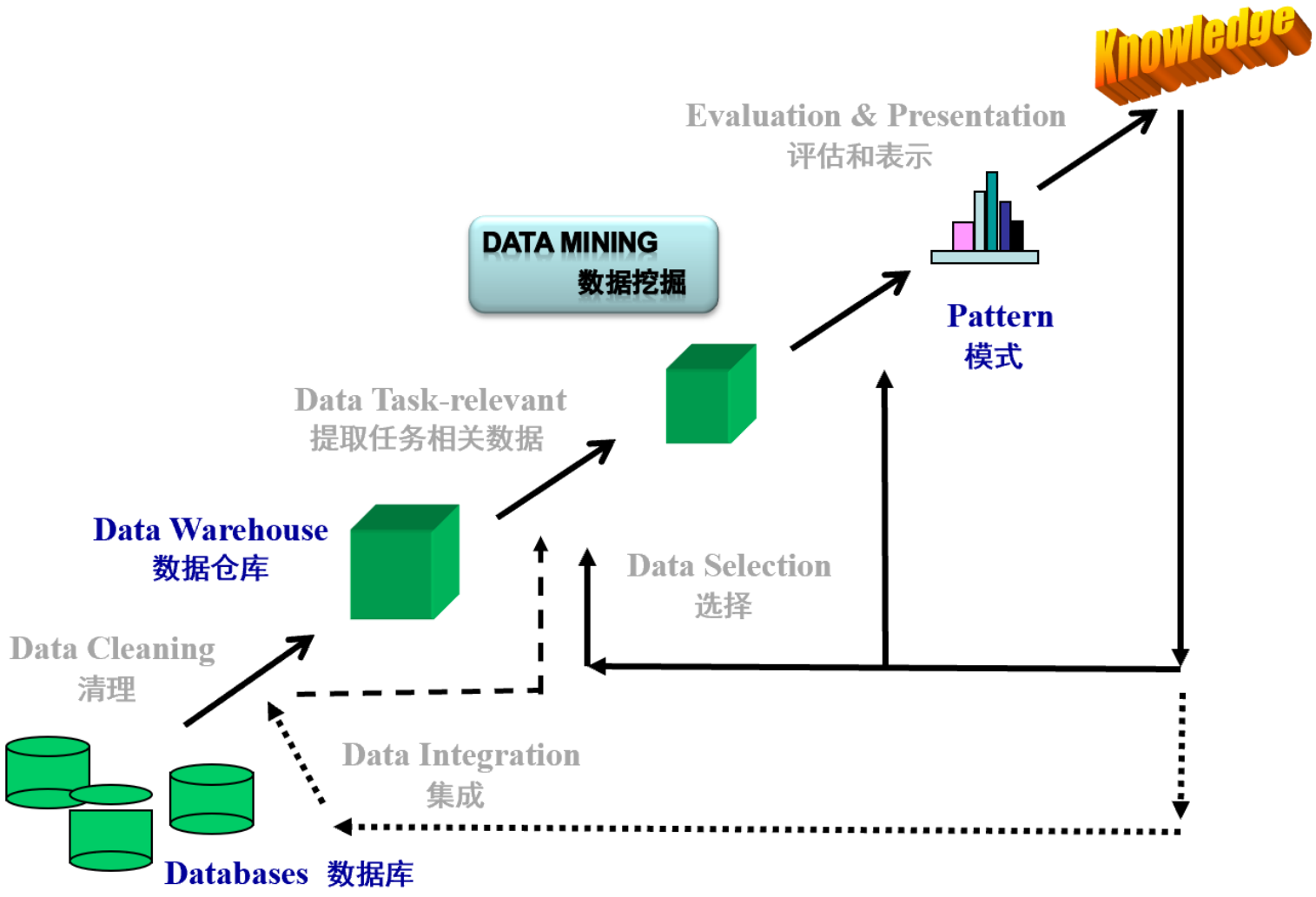
- 数据服务与数据分析的区别
  - 数据服务一般在一个事务中，只需存取少量的数据记录，目的是反映操作型业务对数据的修改，比如新建订单、为订单付款、账户余额的扣减等
  - 数据分析则往往要在大量数据上（可以是整个数据集），进行统计汇总以及更深层次的分析操作，目的是从大量数据中获得少量的指标和分析结果，以便从总体上对数据中隐藏的规律有所认识
- 在用户和数据处理系统进行一次交互的过程中，用户使用到的功能可能囊括数据服务、简单数据分析和复杂数据分析

数据仓库与数据挖掘的关系

- 构造数据仓库涉及数据清理、数据集成和数据变换
  - — 可以看做数据挖掘的一个重要预处理步骤
- 数据仓库泛化、合并多维空间的数据，提供联机分析处理OLAP（Online Analytical Processing） 工具，用于不同粒度的数据分析，有利于在此之上进行关联、分类、聚类等
- 构造数据仓库和OLAP已经成为知识发现过程的基本步骤
  - — 许多其他数据挖掘功能，如关联、分类、预测和聚类，都可以与OLAP操作集成，以加强多个抽象层上的交互知识挖掘。数据仓库已经成为数据分析和联机数据分析处理的日趋重要的平台，并将为数据挖掘提供有效的平台



数据仓库与数据挖掘的关系



Chapter 5-02: Data Warehousing and On-line Analytical Processing

- **Data Warehouse: Basic Concepts**
  - (a) What Is a Data Warehouse?
  - (b) Data Warehouse: A Multi-Tiered Architecture
  - (c) Three Data Warehouse Models: Enterprise Warehouse, Data Mart, ad Virtual Warehouse
  - (d) Extraction, Transformation and Loading
  - (e) Metadata Repository
- **Data Warehouse Modeling: Data Cube and OLAP**
  - (a) Cube: A Lattice of Cuboids
  - (b) Conceptual Modeling of Data Warehouses
  - (c) Stars, Snowflakes, and Fact Constellations: Schemas for Multidimensional Databases
  - (d) Dimensions: The Role of Concept Hierarchy
  - (e) Measures: Their Categorization and Computation
  - (f) Cube Definitions in Database systems
  - (g) Typical OLAP Operations
  - (h) A Starnet Query Model for Querying Multidimensional Databases
- **Data Warehouse Design and Usage**
  - (a) Design of Data Warehouses: A Business Analysis Framework
  - (b) Data Warehouses Design Processes
  - (c) Data Warehouse Usage
  - (d) From On-Line Analytical Processing to On-Line Analytical Mining
- **Data Warehouse Implementation**
  - (a) Efficient Data Cube Computation: Cube Operation, Materialization of Data Cubes, and Iceberg Cubes
  - (b) Indexing OLAP Data: Bitmap Index and Join Index
  - (c) Efficient Processing of OLAP Queries
  - (d) OLAP Server Architectures: ROLAP vs. MOLAP vs. HOLAP
- **Summary**

Chapter 5-02: Data Warehousing and On-line Analytical Processing

- **Data Warehouse: Basic Concepts**
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

What is a Data Warehouse?

什么是 数据仓库？

数据库与数据仓库简史

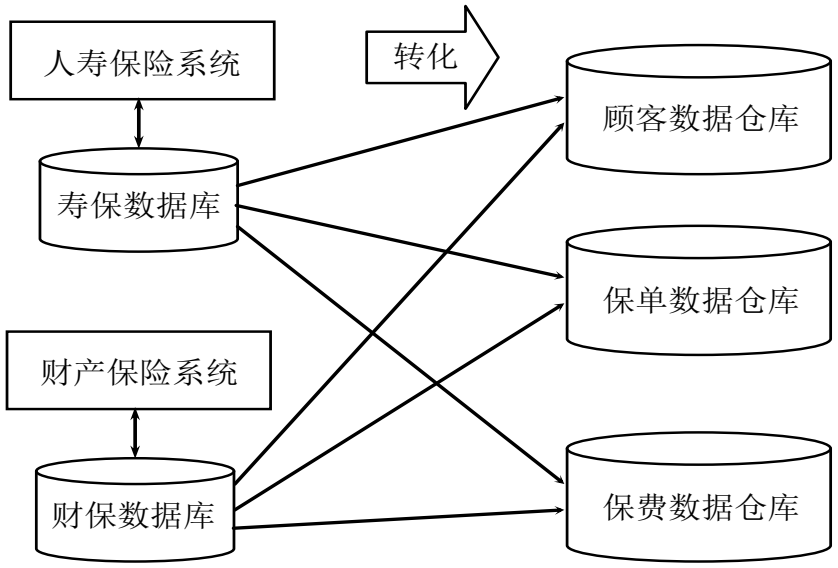
- 20世纪60年代出现了数据库的概念，确立了数据库系统的许多概念、方法和技术
- 70年代由E.F.Codd提出了数据库的关系理模型，开创了数据库关系方法和关系数据理论的研究，为关系数据库技术奠定了理论基础
- 80年代出现成熟的关系数据库管理系统（DBMS）
- 1988年，IBM公司第一次提出了信息仓库（Information Warehouse）的概念，数据仓库初具雏形
- 90年代以后进入数据处理大发展时期，各种数据模型、数据库新技术层出不穷地涌现，如数据仓库和数据挖掘、商务智能、多媒体数据库和Web数据库等
  - 1991年，W.H.Inmon出版了《Building Data Warehouse》一书。W.H.Inmon主张建立数据仓库时采用自上而下方式，以第3范式进行数据仓库模型设计
  - 1993年，R.Kimball（金博尔）出版了《The DataWarehouse Toolkit》一书，认同了比尔恩门对于数据仓库的定义，但对具体的构建方法做了更进一步的研究。R.Kimball主张自下而上的方式，力推数据集市建设
  - .....

What is a Data Warehouse?

- Defined in many different ways, but not rigorously.
  - A decision support database that is maintained separately from the organization's operational database
  - 它是一个提供决策支持功能的数据库，它与公司的操作数据库分开维护
  - Support information processing by providing a solid platform of consolidated, historical data for analysis.
  - 为统一的历史数据分析提供坚实的平台，对信息处理提供支持
- "A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process."—W. H. Inmon
- "数据仓库是一个 面向主题的、集成的、随时间而变化的、不容易丢失的数据集合，支持管理部门的决策过程."— W. H. Inmon
- Data warehousing
  - The process of constructing and using data warehouses

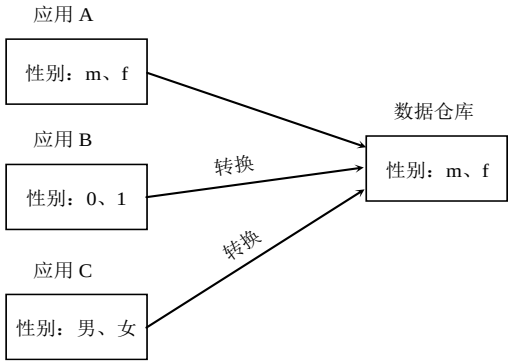
Data Warehouse—Subject-Oriented

- **面向主题，是数据仓库显著区别于关系数据库系统的一个特征**
  - **Organized around major subjects, such as customer, product, sales**
  - **总是围绕一些主题，如顾客、供应商、产品、销售组织等**
  - **Focusing on the modeling and analysis of data for decision makers, not on daily operations or transaction processing**
  - **关注决策者的数据建模与分析，而不是集中于组织机构的日常操作和事务处理**
  - **Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process**
  - **排除对于决策无用的数据，提供特定主题的简明视图**



Data Warehouse—Integrated

- Constructed by integrating multiple, heterogeneous data sources
  - relational databases, flat files, on-line transaction records
- 一个数据仓库是通过 集成多个异种数据源 来构造的
  - E.g., 关系数据库，一般文件，*联机事务处理记录*
- Data cleaning and data integration techniques are applied.
  - Ensure consistency in naming conventions, encoding structures, attribute measures, etc. among different data sources
    - E.g., Hotel price: currency, tax, breakfast covered, etc.
  - When data is moved to the warehouse, it is converted.
- 使用 数据清理 和 数据集成 技术
  - 当数据被移到数据仓库时，它们要经过转化
  - 确保命名约定、编码结构、属性度量等的一致性
    - E.g., Hotel price: currency, tax, breakfast covered, etc.

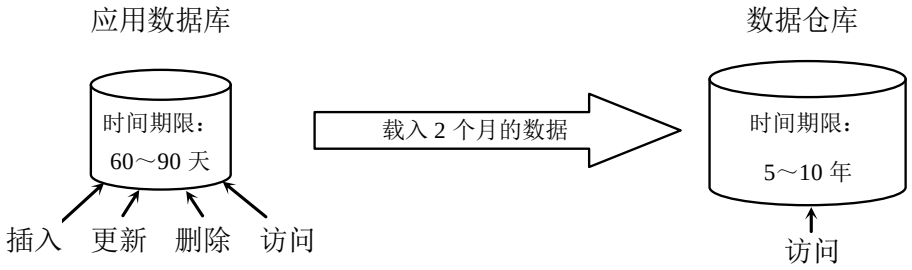


集成

数据仓库中存储的数据一般从企业原来已建立的数据库系统中提取出来，但并不是原有数据的简单拷贝，而是经过了抽取、筛选、清理、转换、综合等工作

Data Warehouse—Time Variant

- The time horizon for the data warehouse is significantly longer than that of operational systems
  - Operational database: current value data
  - Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)
- 数据仓库是从历史的角度提供信息，时间范围比操作数据库系统要长的多
  - 数据仓库:从历史的角度提供信息（比如过去 5-10 年）
  - 操作数据库系统: 主要保存当前数据
- Every key structure in the data warehouse
  - Contains an element of time, explicitly or implicitly
  - But the key of operational data may or may not contain “time element”
- 数据仓库的关键结构
  - 数据仓库中每一个关键结构都隐式或显式地包含时间元素
  - 操作数据库中的关键结构可能不包括时间元素



随时间而变化即时变的

数据仓库大多关注的是历史数据，其中数据是批量载入的，即定期从操作型应用系统中接收新的数据内容，这使得数据仓库中的数据总是拥有时间维度

Data Warehouse—Nonvolatile

- A physically separate store of data transformed from the operational environment
- 尽管数据仓库中的数据来自于操作数据库，但数据仓库总是在物理上分离保存的
- Operational update of data does not occur in the data warehouse environment
  - Does not require transaction processing, recovery, and concurrency control mechanisms
  - Requires only two operations in data accessing:
    - *initial loading of data* and *access of data*
- 操作数据库的 更新操作 不会出现在数据仓库环境下
  - 不需要事务处理、恢复和并发控制等机制
  - 只需要两种数据访问:
    - Initial loading of data 数据的初始装载
    - Access of data 数据访问



稳定性即非易失的

操作型数据库系统中一般只存储短期数据，因此其数据是不稳定的，它记录的是系统中数据变化的瞬态。操作型数据库要经常进行修改、添加

数据仓库中数据大多表示过去某一时刻的数据，主要用于查询、分析，不需要事务处理、恢复和并发控制等机制，只需进行数据装载和访问

“为什么不直接在传统数据库上进行联机分析处理，
而是另外花费时间和资源去构造分离的数据仓库？”

OLTP vs. OLAP

“数据仓库与操作数据库系统有什么区别？”

Why a Separate Data Warehouse?

- High performance for both systems
  - DBMS— tuned for OLTP: access methods, indexing, concurrency control, recovery
  - Warehouse—tuned for OLAP: complex OLAP queries, multidimensional view, consolidation
- Different functions and different data:
  - missing data: Decision support requires historical data which operational DBs do not typically maintain
  - data consolidation: DS requires consolidation (aggregation, summarization) of data from heterogeneous sources
  - data quality: different sources typically use inconsistent data representations, codes and formats which have to be reconciled
- Note: There are more and more systems which perform OLAP analysis directly on relational databases

Why a Separate Data Warehouse?
为什么需要一个分离的数据仓库?

- *“为什么不直接在传统数据库上进行联机分析处理，而是另外花费时间和资源去构造分离的数据仓库？”*
- 提高两个系统的性能
  - DBMS是为OLTP而设计的：存储方式,索引, 并发控制, 恢复
  - 数据仓库是为OLAP而设计：复杂的 OLAP查询, 多维视图，汇总
- 不同的功能和不同的数据:
  - 历史数据: 决策支持需要历史数据，而这些数据在操作数据库中一般不会去维护
  - 数据汇总：决策支持需要将来自异种源的数据统一（如聚集和汇总）
  - 数据质量: 不同的源使用不一致的数据表示、编码和格式，对这些数据进行有效的分析需要将他们转化后进行集成

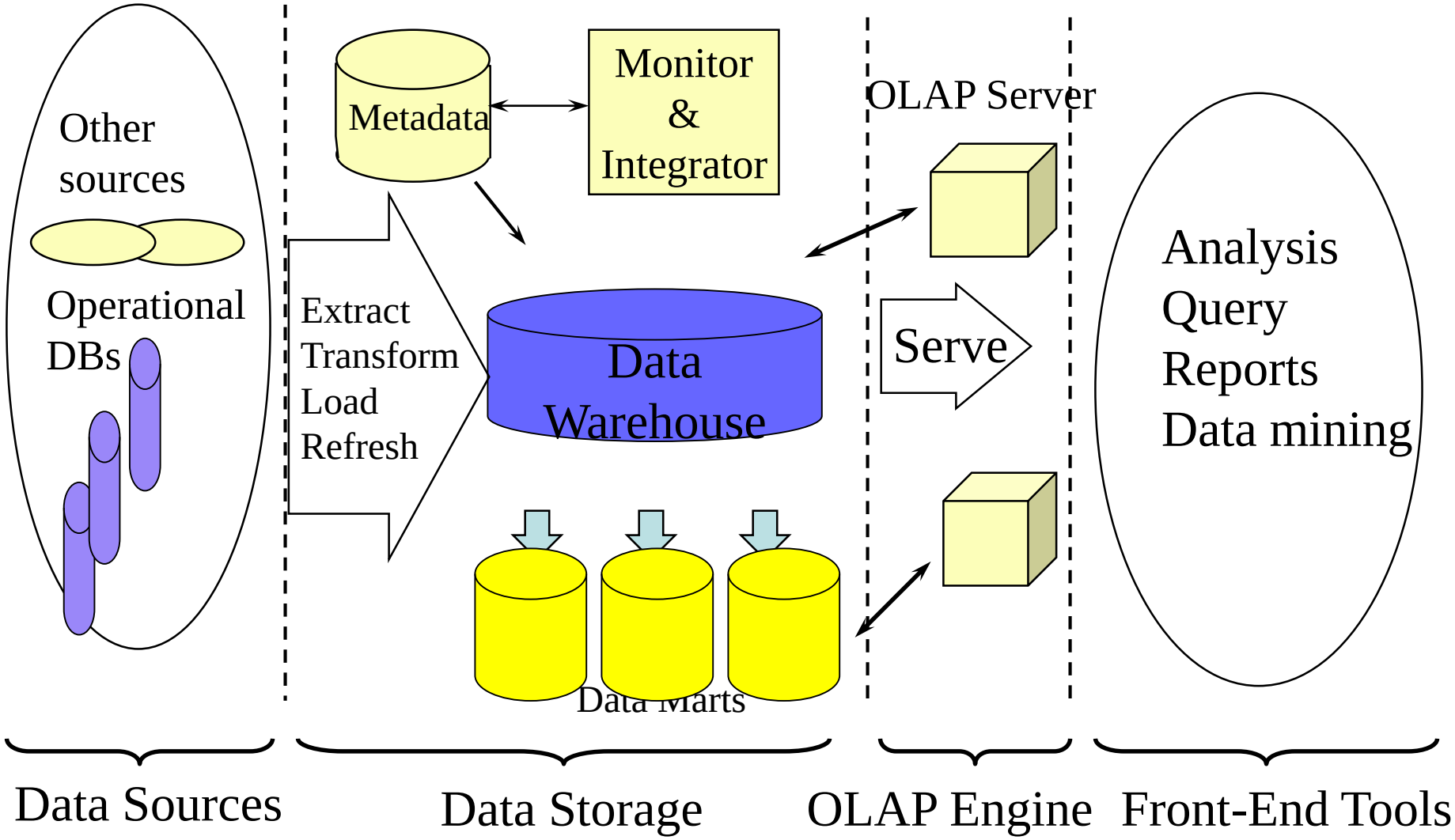
OLTP vs. OLAP

	OLTP	OLAP
users	clerk, IT professional	knowledge worker
function	day to day operations	decision support
DB design	application-oriented	subject-oriented
data	current, up-to-date detailed, flat relational isolated	historical, summarized, multidimensional integrated, consolidated
usage	repetitive	ad-hoc
access	read/write index/hash on prim. key	lots of scans
unit of work	short, simple transaction	complex query
# records accessed	tens	millions
#users	thousands	hundreds
DB size	100MB-GB	100GB-TB
metric	transaction throughput	query throughput, response

数据仓库是一种多层体系结构

Data Warehouse: A Multi-Tiered Architecture

Data Warehouse: A Multi-Tiered Architecture



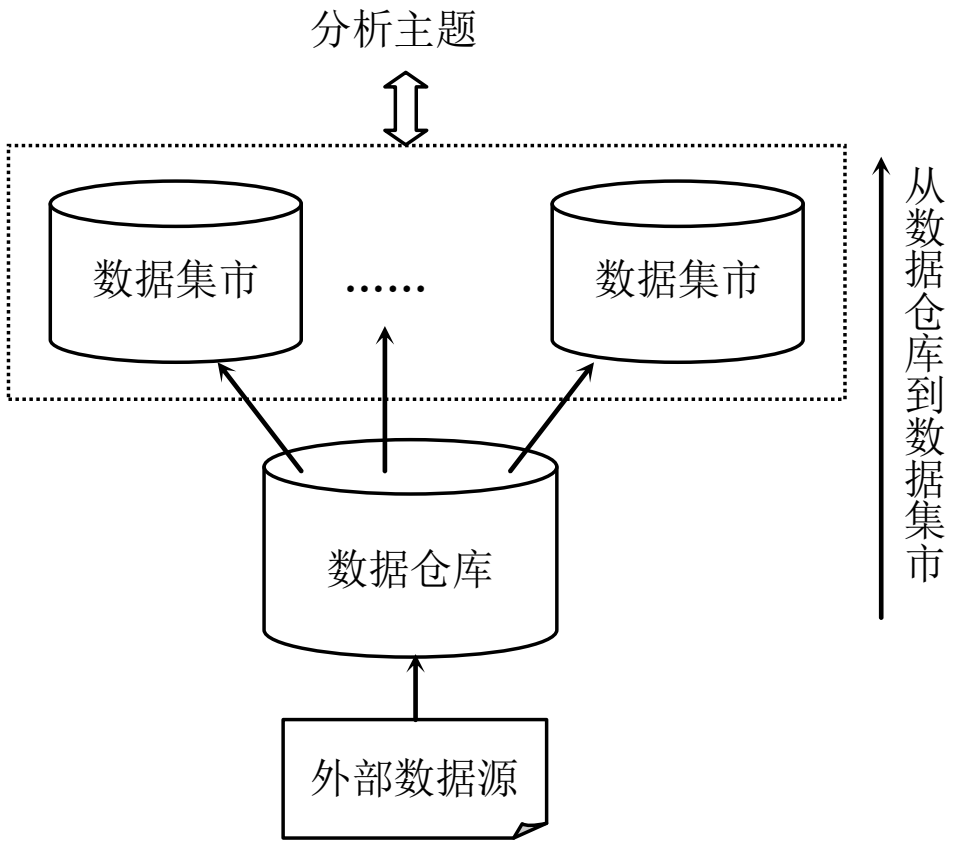
Three Data Warehouse Models

- **Enterprise warehouse**
  - —collects all of the information about subjects spanning the entire organization
- **Data Mart**
  - —a subset of corporate-wide data that is of value to a specific groups of users. Its scope is confined to specific, selected groups, such as marketing data mart
    - Independent vs. dependent (directly from warehouse) data mart
- **Virtual warehouse**
  - —A set of views over operational databases
  - —Only some of the possible summary views may be materialized

Extraction, Transformation, and Loading (ETL)

- Data extraction
  - —get data from multiple, heterogeneous, and external sources
- Data cleaning
  - —detect errors in the data and rectify them when possible
- Data transformation
  - —convert data from legacy or host format to warehouse format
- Load
  - —sort, summarize, consolidate, compute views, check integrity, and build indicies and partitions
- Refresh
  - —propagate the updates from the data sources to the warehouse

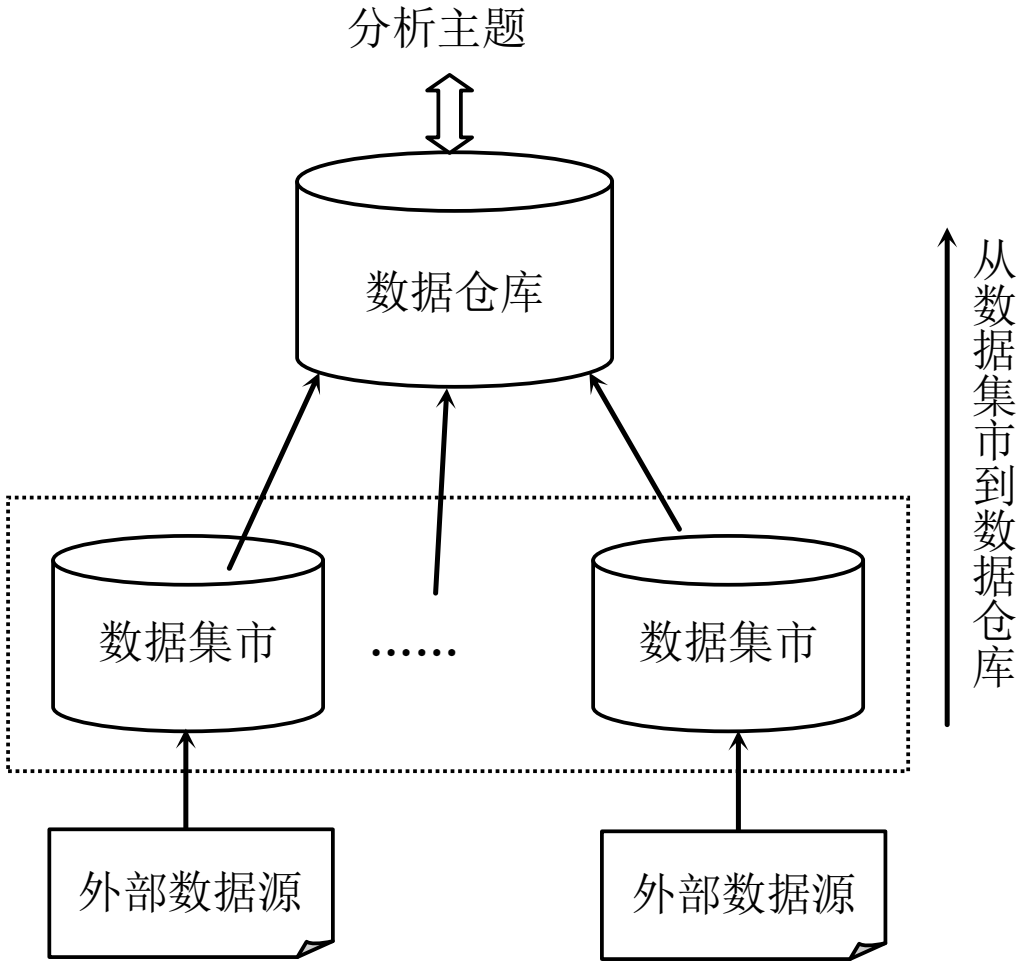
数据仓库先整体再局部的构建模式：W.H.Inmon模式



优点：
数据规范化程度高，最小化数据冗余与不一致性；便于全局数据的分析和挖掘

缺点：
建设周期长、见效慢；风险程度相对大

数据仓库先局部再整体的构建模式：Ralph Kimball 模式



优点：
投资少、见效快；在设计上相对灵活；易于实现
缺点：
会有一定级别的冗余和不一致性

数据仓库开发：自顶向下 V.S. 自底向上

数据仓库开发上的困难

—自顶向下方法

- 从全系统的角度提供解决方案，使得（模块）集成的问题最小；但是该方法十分昂贵，需要对组织进行长期研究和建模分析。

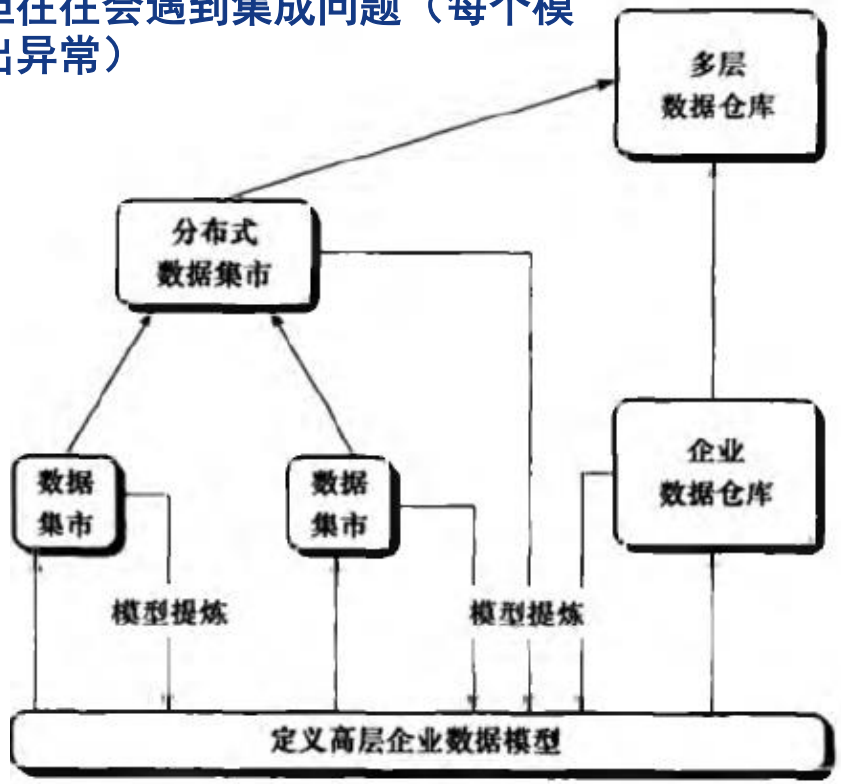
—自底向上方法

- 提供了更多的开发灵活性，价格便宜；但往往会遇到集成问题（每个模块单独运行都没有问题，但是一集成就出异常）

解决方法：

—使用递增性、演化性的开发方法

- 高层数据模型
- 企业仓库和数据集市并行开发
- 通过分布式模型集成各数据集市
- 多层数据仓库



建立数据仓库看成构建和使用数据仓库的过程

- 数据仓库的构建包括一系列的数据预处理过程
  - —数据清理
  - —数据集成
  - —数据变换
- 数据仓库的使用热点是商业决策行为，例如：
  - —增加客户聚焦
  - —产品重定位
  - —寻找获利点
  - —客户关系管理

元数据库

Metadata Repository

Metadata Repository

- Meta data is the data defining warehouse objects. It stores:
- Description of the structure of the data warehouse
  - schema, view, dimensions, hierarchies, derived data defn, data mart locations and contents
- Operational meta-data
  - data lineage (history of migrated data and transformation path), currency of data (active, archived, or purged), monitoring information (warehouse usage statistics, error reports, audit trails)
- The algorithms used for summarization
- The mapping from operational environment to the data warehouse
- Data related to system performance
  - warehouse schema, view and derived data definitions
- Business data
  - business terms and definitions, ownership of data, charging policies

Chapter 5-02: Data Warehousing and On-line Analytical Processing

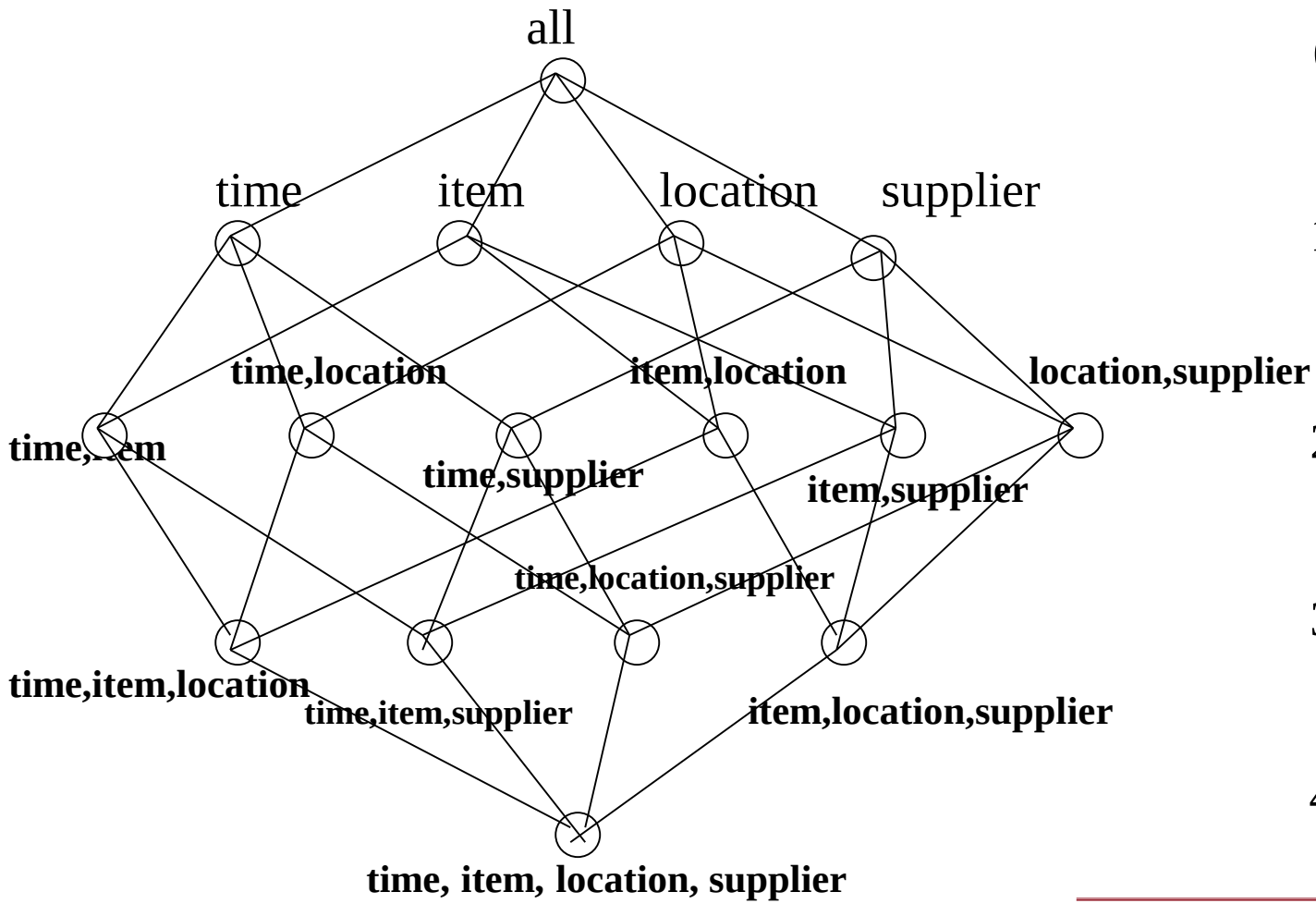
- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- **Data Cube and OLAP**
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

From Tables and Spreadsheets to Data Cubes

- A data warehouse is based on a multidimensional data model which views data in the form of a data cube
- A data cube, such as sales, allows data to be modeled and viewed in multiple dimensions
  - Dimension tables, such as item (item_name, brand, type), or time(day, week, month, quarter, year)
  - Fact table contains measures (such as dollars_sold) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a base cuboid. The top most 0-D cuboid, which holds the highest-level of summarization, is called the apex cuboid. The lattice of cuboids forms a data cube.

Cube: A Lattice of Cuboids
立方体：一个方体格

- 数据仓库语义中，一个n-D 底层方体称为**基本方体**；最高层的0-D方体，存放最高层的汇总，称为**顶点方体**；所有的方体格组成了**数据立方体**



0-D (*apex*) cuboid

1-D cuboids

2-D cuboids

3-D cuboids

4-D (*base*) cuboid

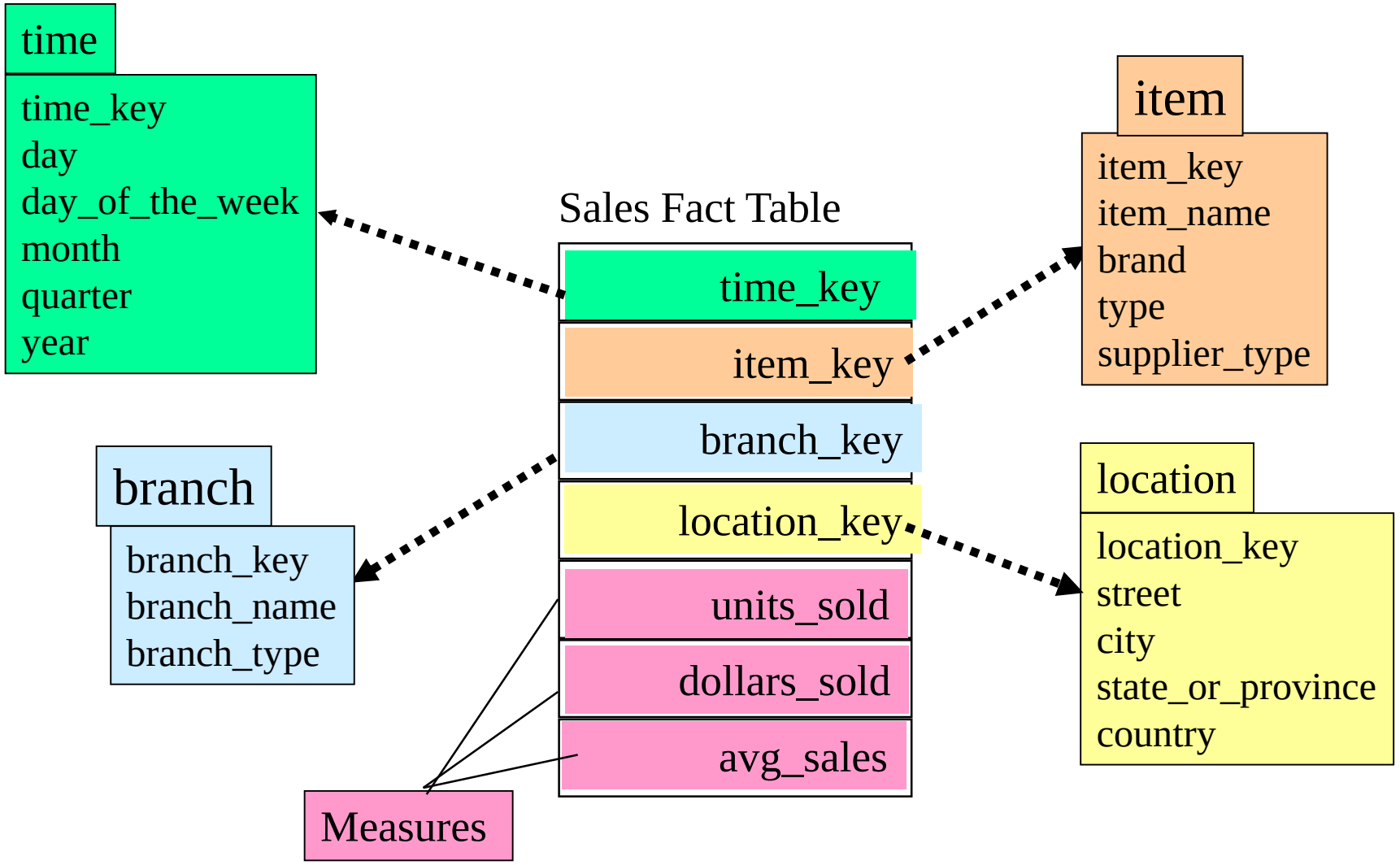
Conceptual Modeling of Data Warehouses

- Modeling data warehouses: dimensions & measures
  - Star schema: A fact table in the middle connected to a set of dimension tables
  - Snowflake schema: A refinement of star schema where some dimensional hierarchy is normalized into a set of smaller dimension tables, forming a shape similar to snowflake
  - Fact constellations: Multiple fact tables share dimension tables, viewed as a collection of stars, therefore called galaxy schema or fact constellation

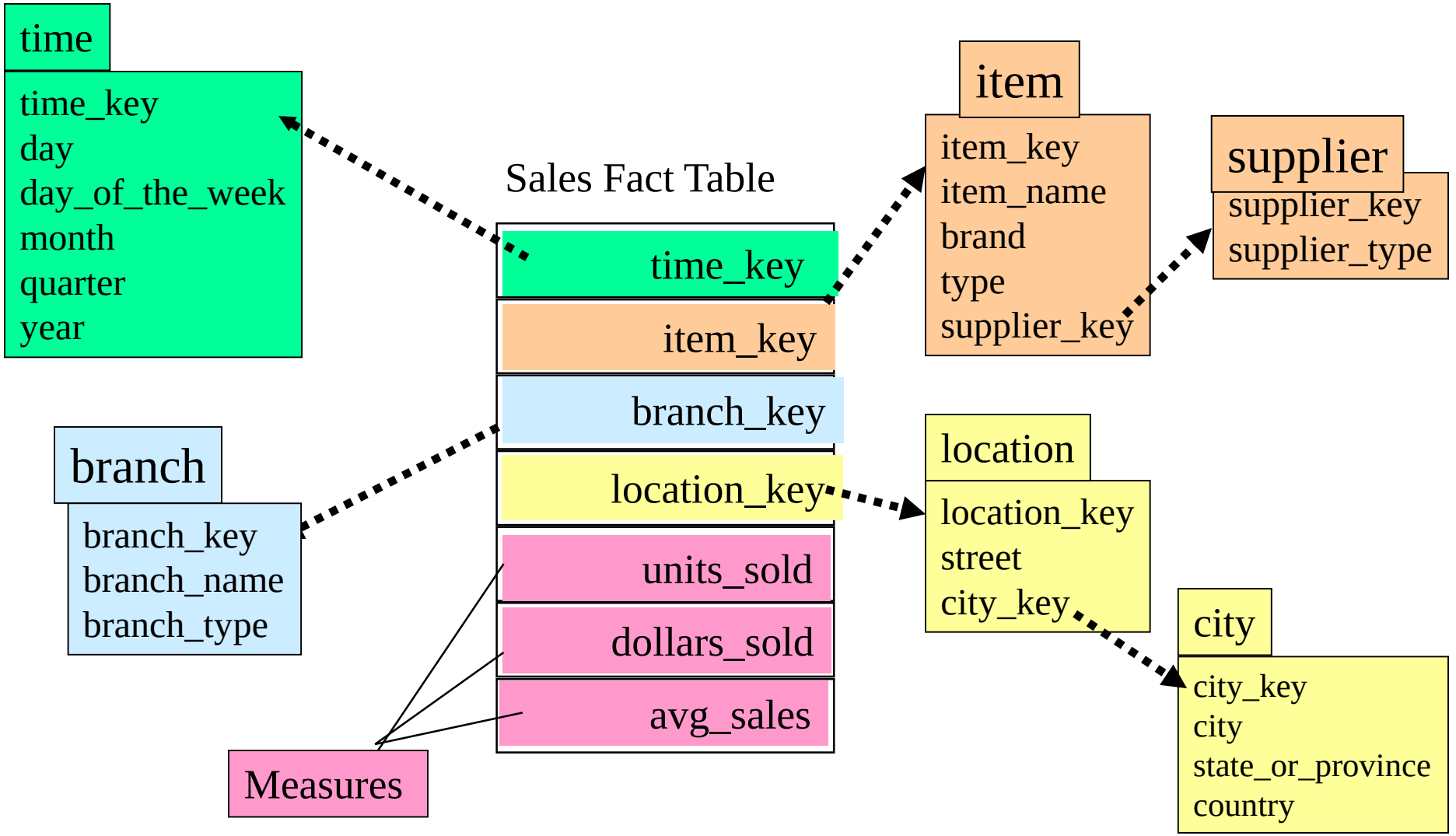
数据仓库的概念模型

- **最流行的数据仓库概念模型是多维数据模型。这种模型可以以星型模式、雪花模式、或事实星座模式的形式存在。**
  - —星型模式(Star schema): 事实表在中心，周围围绕地连接着维表（每维一个），事实表含有大量数据，没有冗余
  - —雪花模式(Snowflake schema): 是星型模式的变种，其中某些维表是规范化的，因而把数据进一步分解到附加表中。结果，模式图形成类似于雪花的形状
  - —事实星座模式(Fact constellations): 多个事实表共享维表，这种模式可以看作星型模式集，因此称为星系模式(galaxy schema)，或者事实星座(fact constellation)

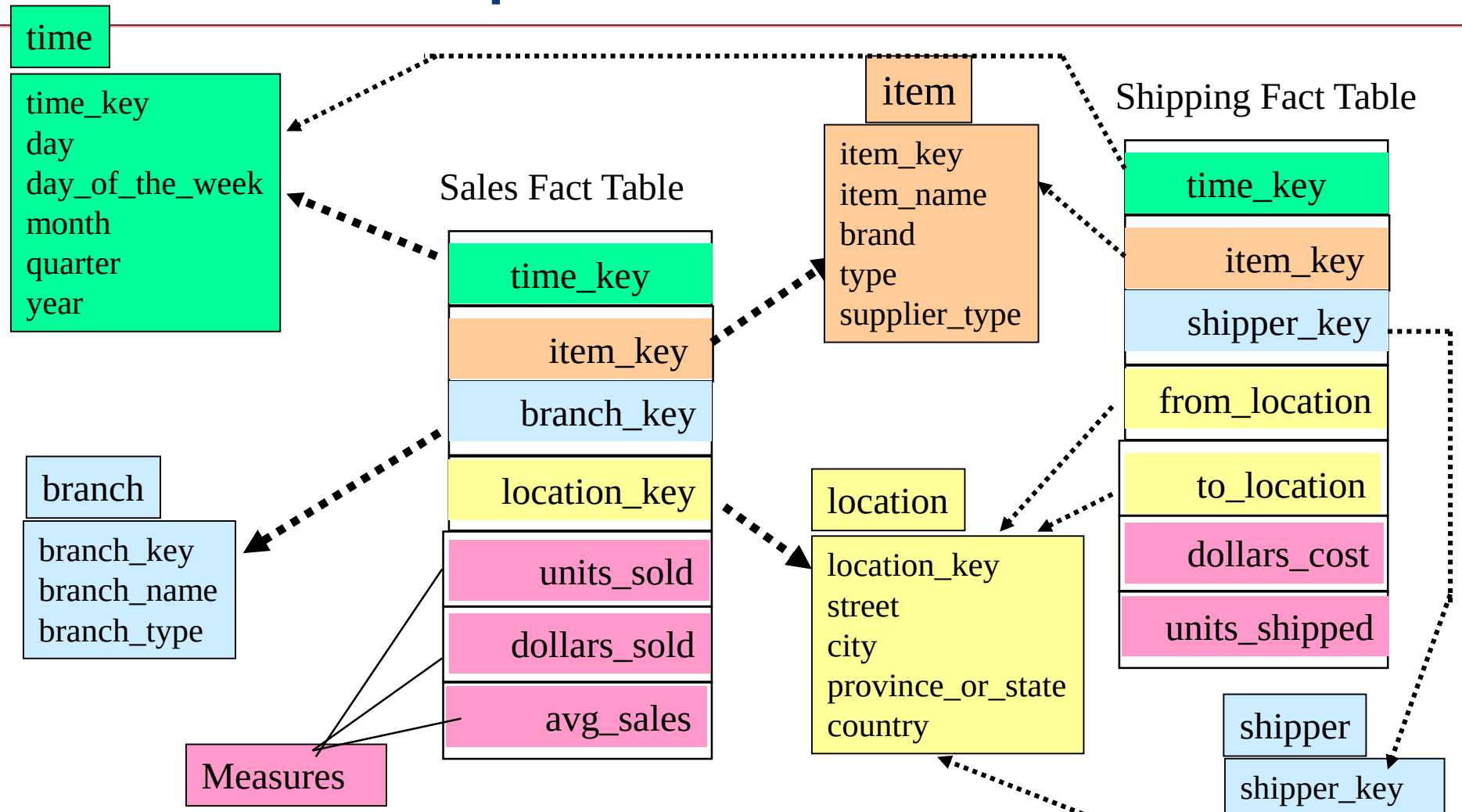
Star Schema


time
time_key
day
day_of_the_week
month
quarter
year
Sales Fact Table
time_key
item_key
branch_key
location_key
units_sold
dollars_sold
avg_sales
item
item_key
item_name
brand
type
supplier_type
branch
branch_key
branch_name
branch_type
location
location_key
street
city
state_or_province
country
Measures

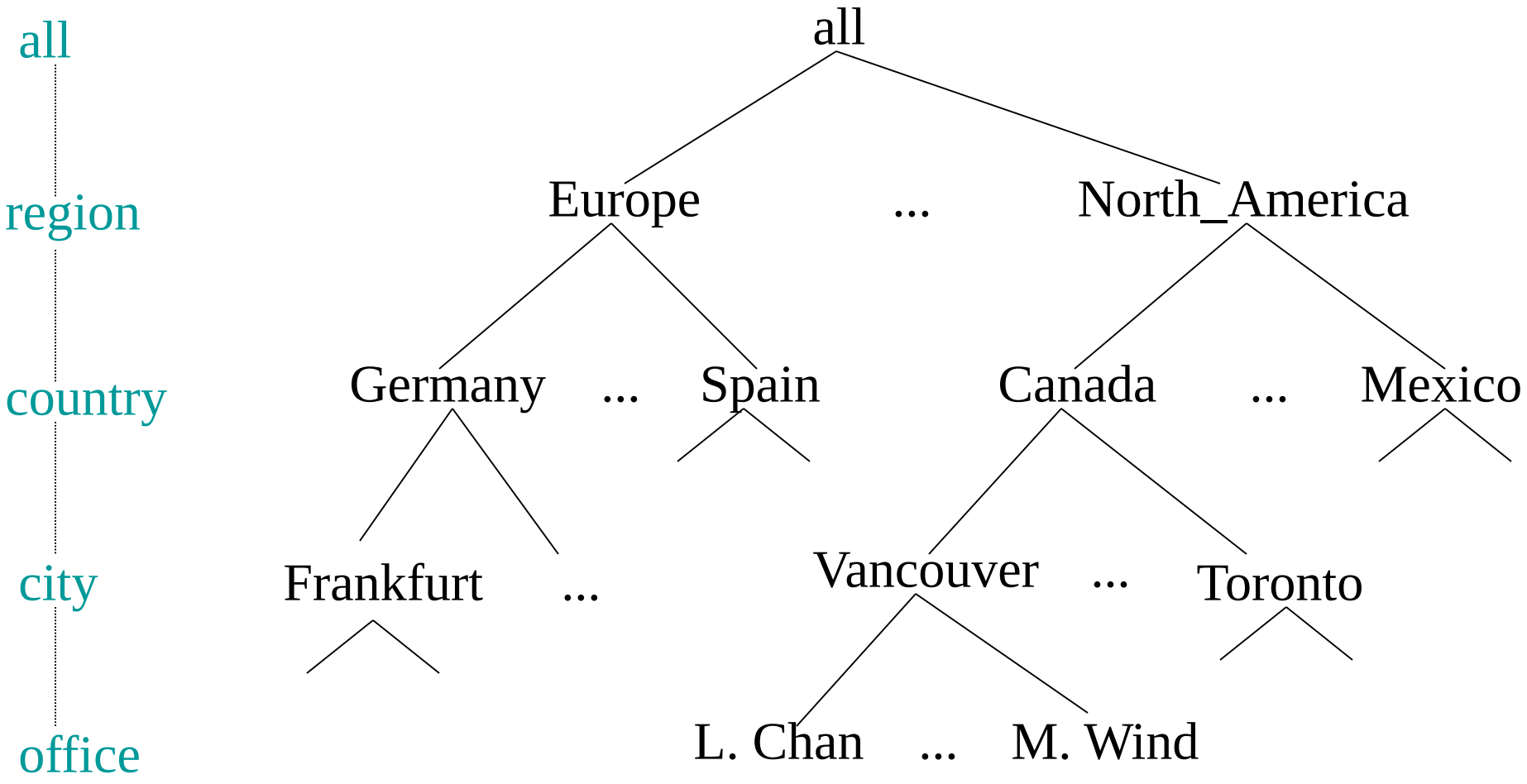
Snowflake Schema



Example of Fact Constellation



A Concept Hierarchy: Dimension (location)

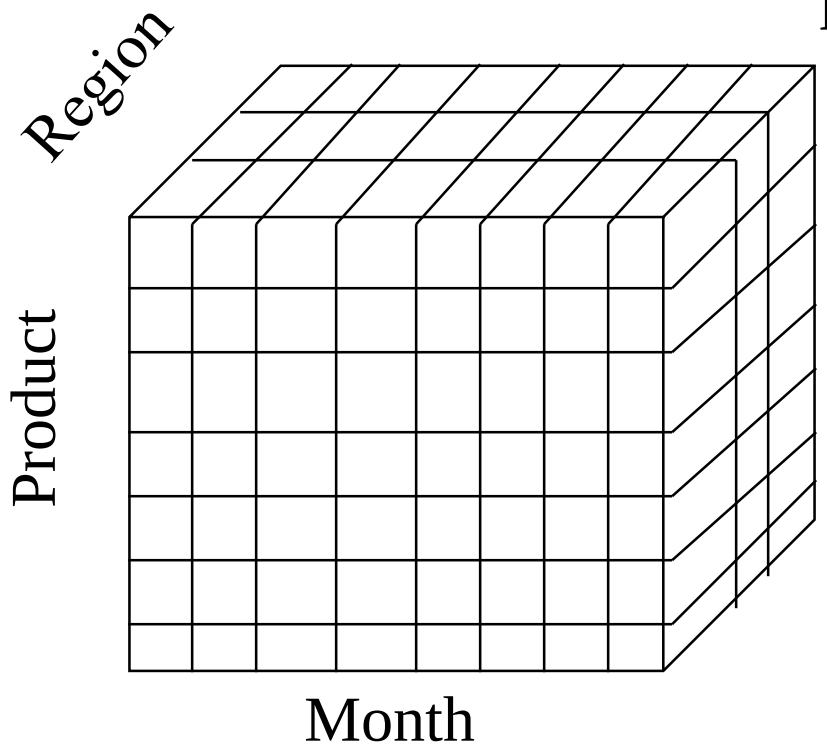


Data Cube Measures: Three Categories

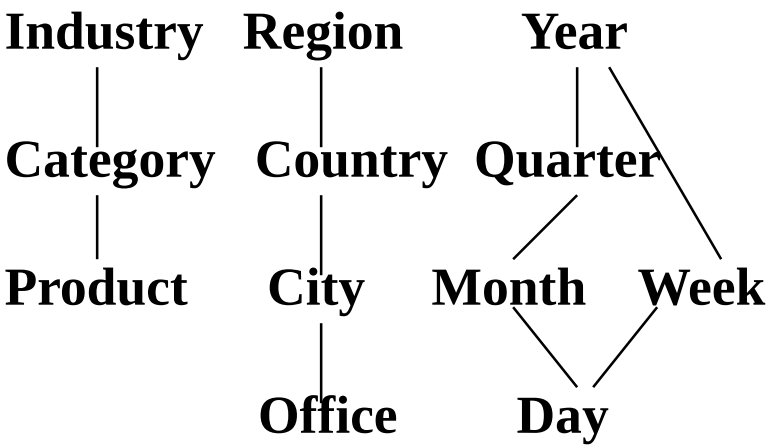
- **Distributive**: if the result derived by applying the function to *n* aggregate values is the same as that derived by applying the function on all the data without partitioning
  - E.g., count(), sum(), min(), max()
- **Algebraic**: if it can be computed by an algebraic function with *M* arguments (where *M* is a bounded integer), each of which is obtained by applying a distributive aggregate function
  - E.g., avg(), min_N(), standard_deviation()
- **Holistic**: if there is no constant bound on the storage size needed to describe a subaggregate.
  - E.g., median(), mode(), rank()

Multidimensional Data

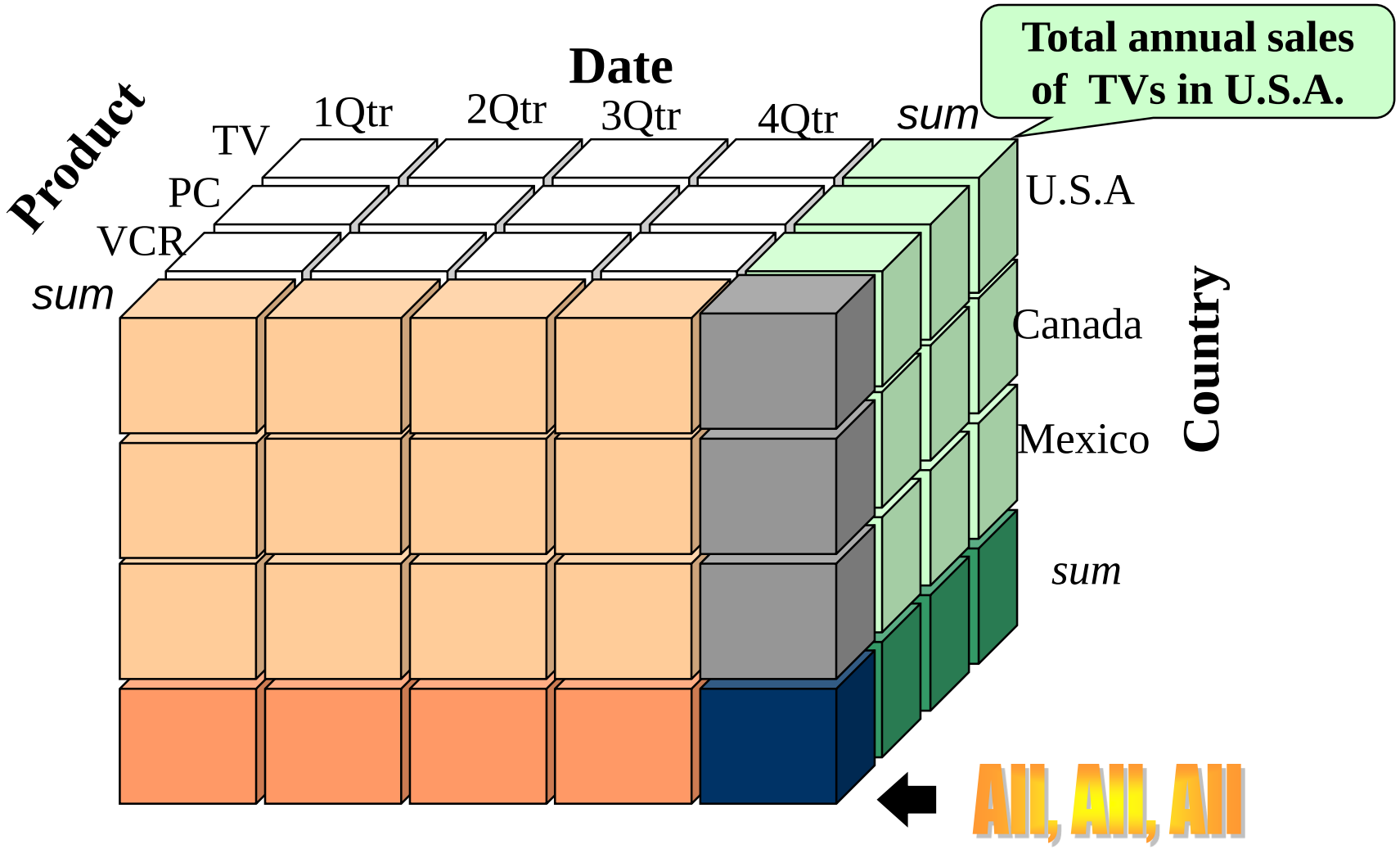
- **Sales volume as a function of product, month, and region**



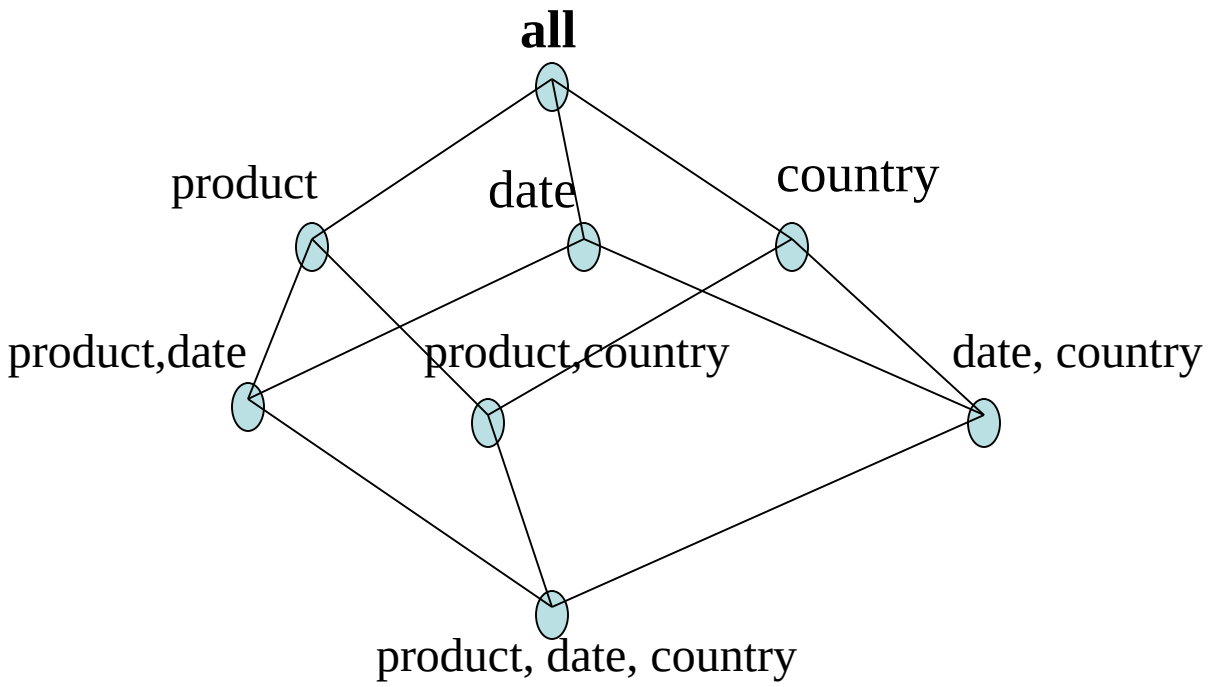
Dimensions: ***Product, Location, Time***
Hierarchical summarization paths



A Sample Data Cube



Cuboids Corresponding to the Cube



0-D (*apex*) cuboid

1-D cuboids

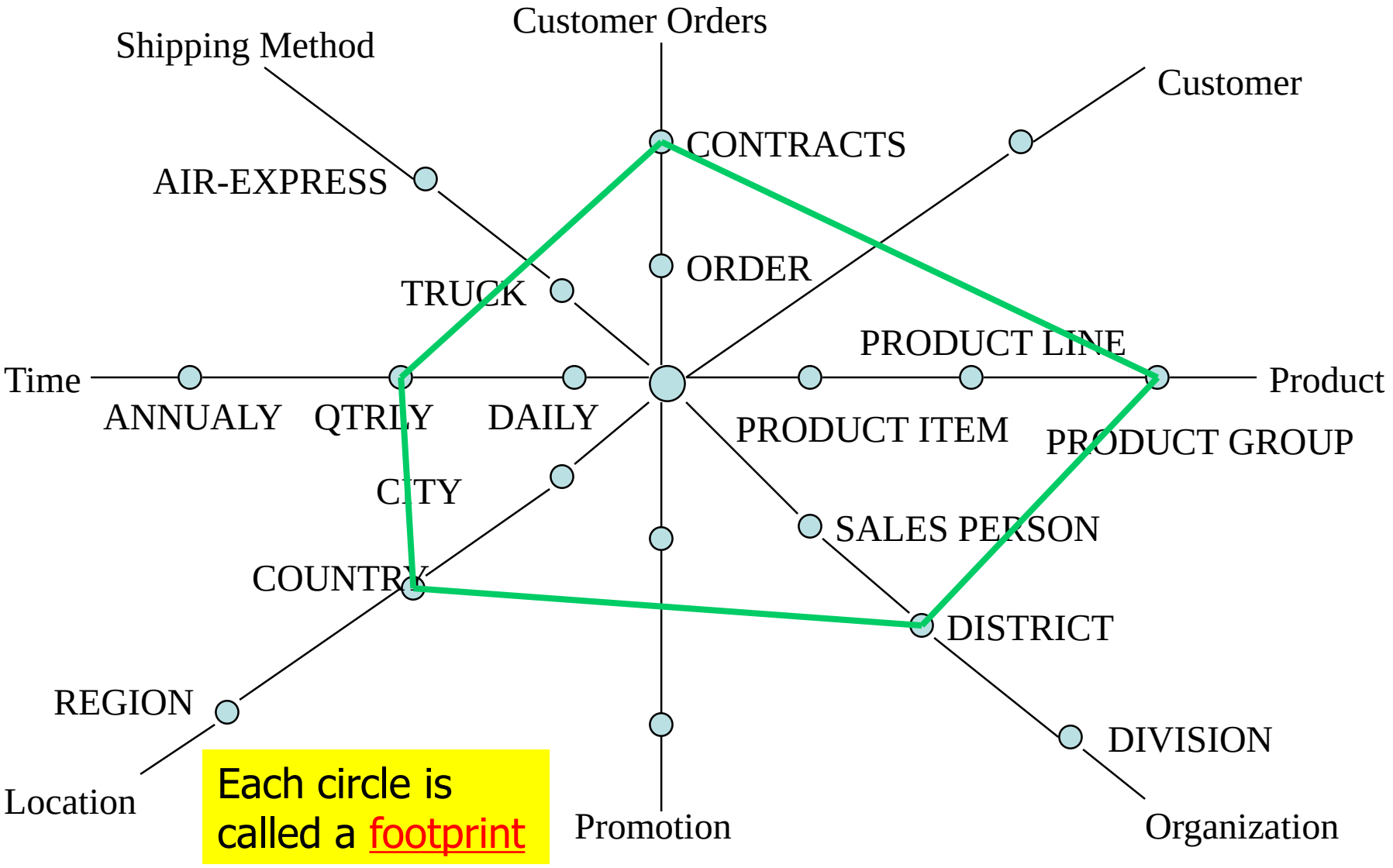
2-D cuboids

3-D (*base*) cuboid

Typical OLAP Operations

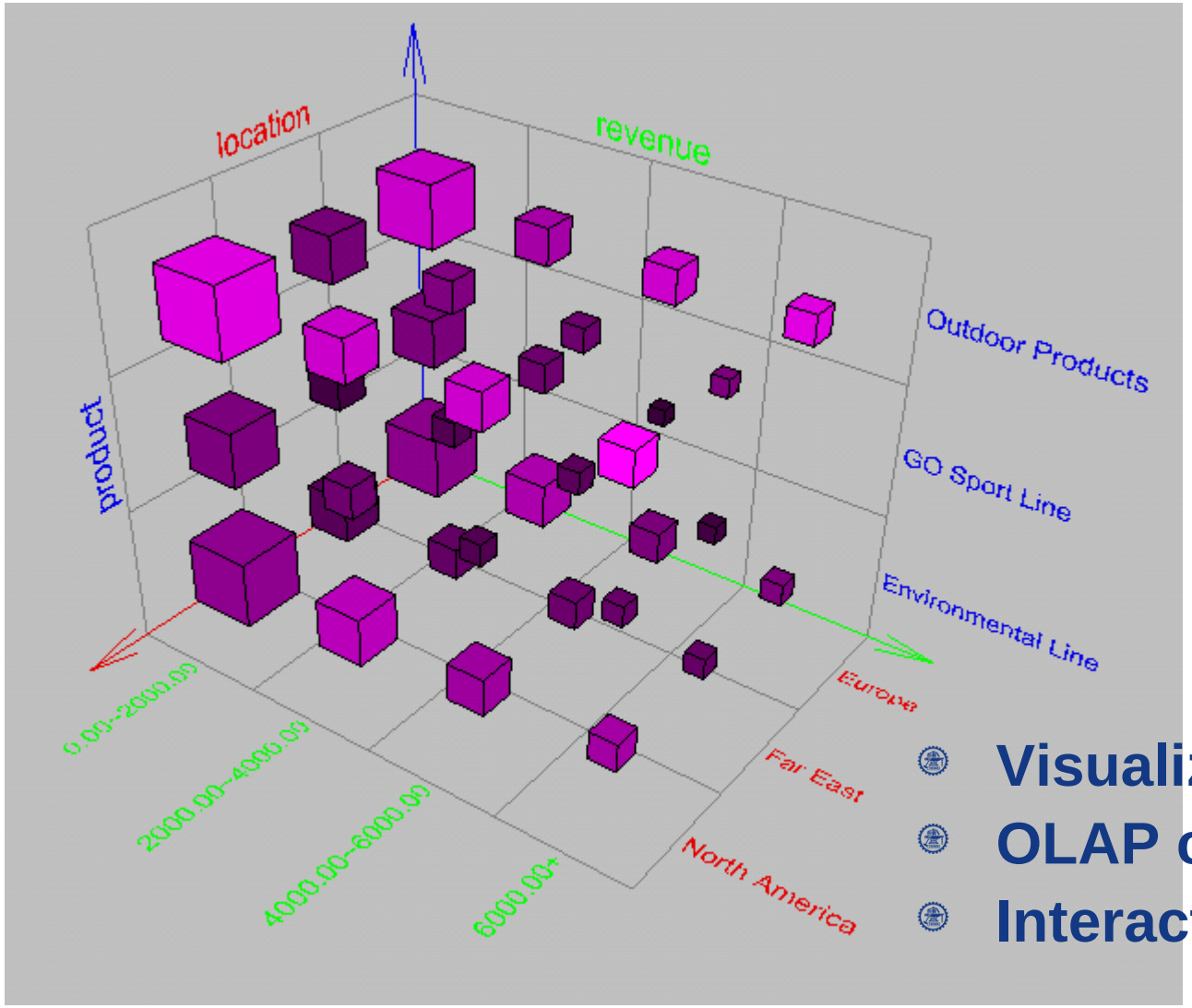
- **Roll up (drill-up):** summarize data
  - *by climbing up hierarchy or by dimension reduction*
- **Drill down (roll down):** reverse of roll-up
  - *from higher level summary to lower level summary or detailed data, or introducing new dimensions*
- **Slice and dice:** *project and select*
- **Pivot (rotate):**
  - *reorient the cube, visualization, 3D to series of 2D planes*
- **Other operations**
  - *drill across: involving (across) more than one fact table*
  - *drill through: through the bottom level of the cube to its back-end relational tables (using SQL)*

A Star-Net Query Model



Each circle is called a footprint

Browsing a Data Cube



- Visualization
- OLAP capabilities
- Interactive manipulation

Chapter 5-02: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- **Data Warehouse Design and Usage**
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

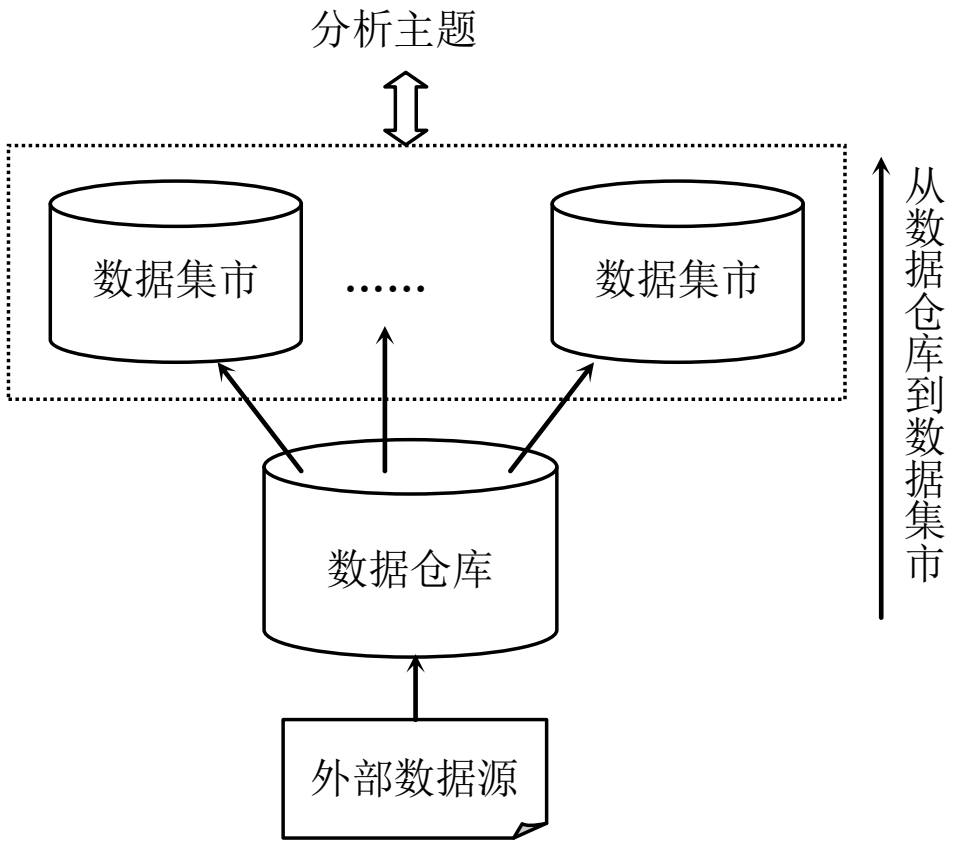
Design of Data Warehouse: A Business Analysis Framework

- **Four views regarding the design of a data warehouse**
  - **—Top-down view**
    - allows selection of the relevant information necessary for the data warehouse
  - **—Data source view**
    - exposes the information being captured, stored, and managed by operational systems
  - **—Data warehouse view**
    - consists of fact tables and dimension tables
  - **—Business query view**
    - sees the perspectives of data in the warehouse from the view of end-user

Data Warehouse Design Process

- Top-down, bottom-up approaches or a combination of both
  - Top-down: Starts with overall design and planning (mature)
  - Bottom-up: Starts with experiments and prototypes (rapid)
- From software engineering point of view
  - Waterfall: structured and systematic analysis at each step before proceeding to the next
  - Spiral: rapid generation of increasingly functional systems, short turn around time, quick turn around
- Typical data warehouse design process
  - Choose a business process to model, e.g., orders, invoices, etc.
  - Choose the *grain* (*atomic level of data*) of the business process
  - Choose the dimensions that will apply to each fact table record
  - Choose the measure that will populate each fact table record

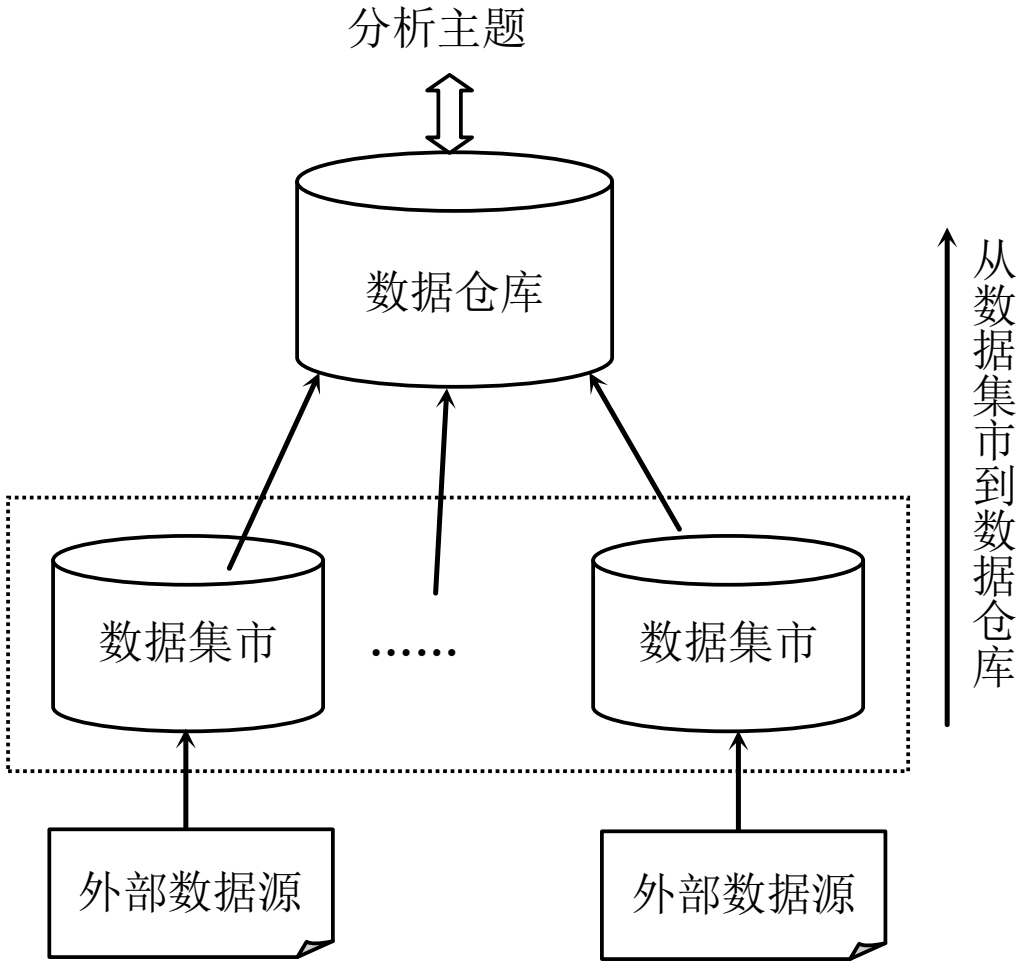
先整体再局部的构建模式：W.H. Inmon模式



优点：
数据规范化程度高，最小化数据冗余与不一致性；便于全局数据的分析和挖掘

缺点：
建设周期长、见效慢；风险程度相对大

先局部再整体的构建模式：Ralph Kimball 模式



优点：
投资少、见效快；在设计上相对灵活；易于实现
缺点：
会有一定级别的冗余和不一致性

数据仓库开发：自顶向下 V.S. 自底向上

数据仓库开发上的困难

—自顶向下方法

- 从全系统的角度提供解决方案，使得（模块）集成的问题最小；但是该方法十分昂贵，需要对组织进行长期研究和建模分析。

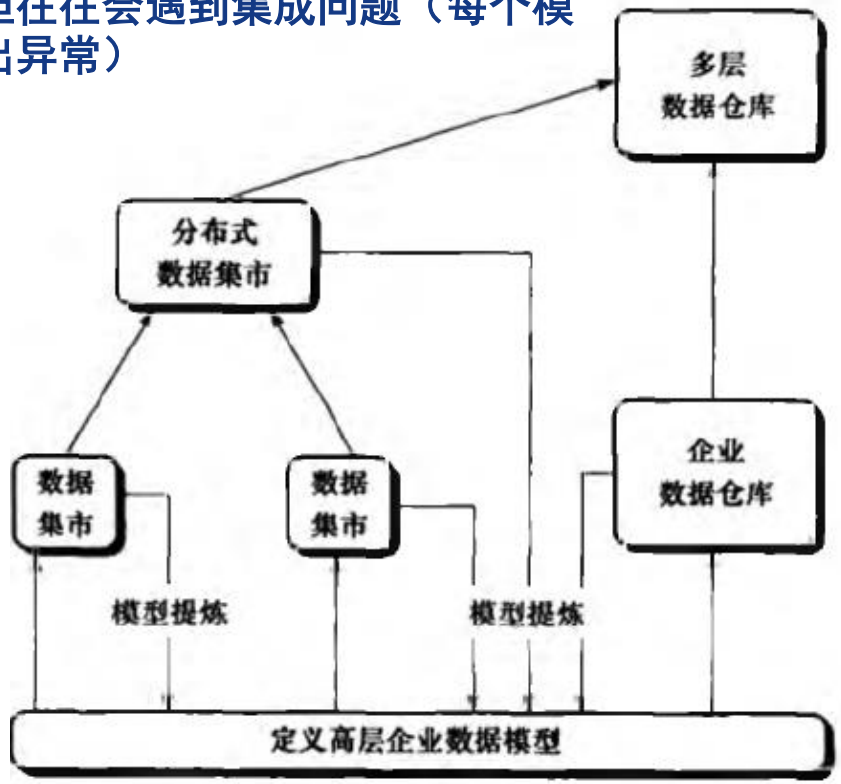
—自底向上方法

- 提供了更多的开发灵活性，价格便宜；但往往会遇到集成问题（每个模块单独运行都没有问题，但是一集成就出异常）

解决方法：

—使用递增性、演化性的开发方法

- 高层数据模型
- 企业仓库和数据集市并行开发
- 通过分布式模型集成各数据集市
- 多层数据仓库



Data Warehouse Usage

- Three kinds of data warehouse applications
  - —Information processing
    - supports querying, basic statistical analysis, and reporting using crosstabs, tables, charts and graphs
  - —Analytical processing
    - multidimensional analysis of data warehouse data
    - supports basic OLAP operations, slice-dice, drilling, pivoting
  - —Data mining
    - knowledge discovery from hidden patterns
    - supports associations, constructing analytical models, performing classification and prediction, and presenting the mining results using visualization tools

Three Application of Data Warehouse
数据仓库的应用

- 数据仓库的三种应用
  - —信息处理
    - 支持查询和基本的统计分析，并使用交叉表、表、图标和图进行报表处理
  - —分析处理
    - 对数据仓库中的数据进行多维数据分析
    - 支持基本的OLAP操作，切块、切片、上卷、下钻、转轴等
  - —数据挖掘
    - 从隐藏模式中发现知识
    - 支持关联分析，构建分析性模型，分类和预测，并用可视化工具呈现挖掘的结果

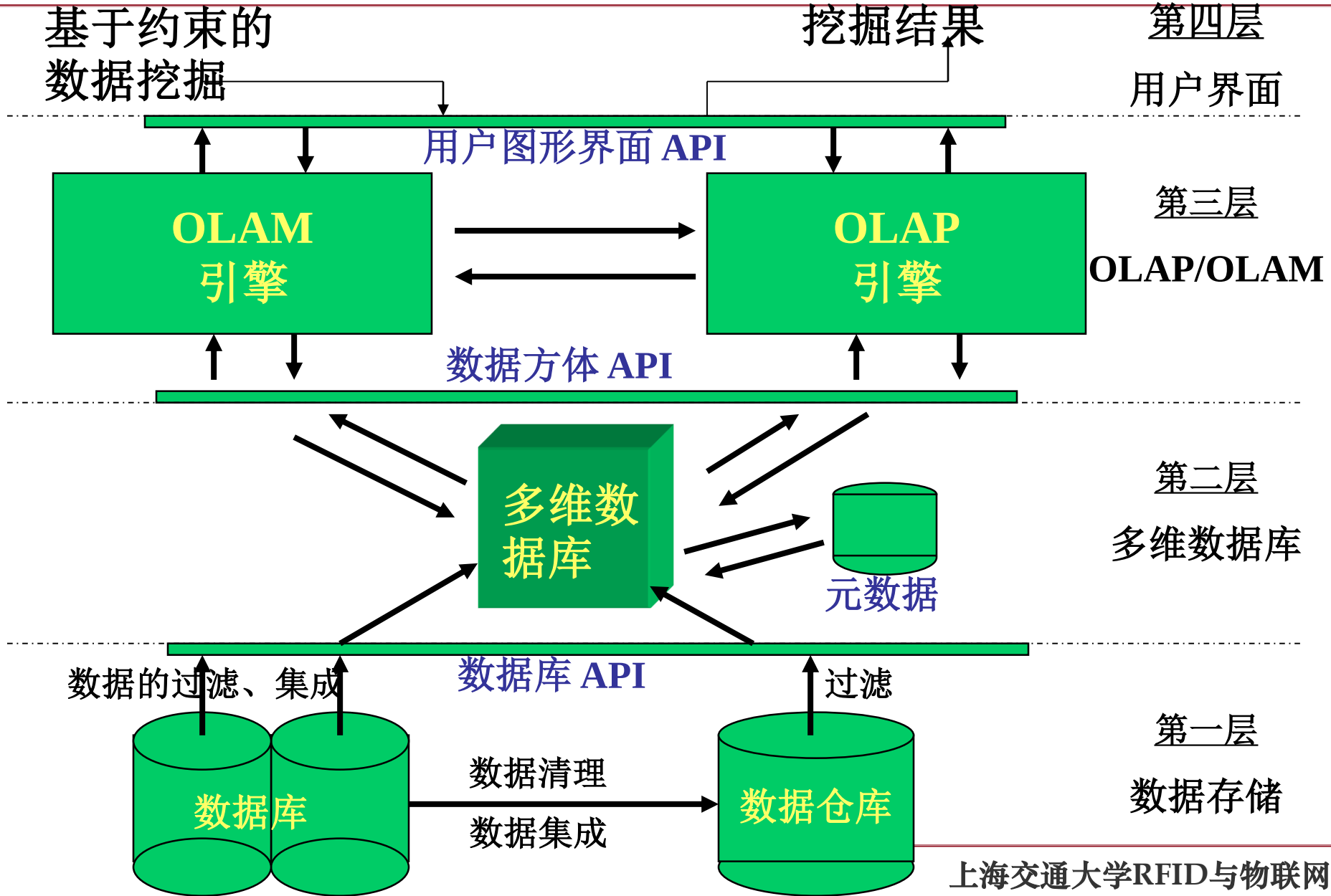
From On-Line Analytical Processing (OLAP) to On Line Analytical Mining (OLAM)

- Why online analytical mining?
  - —High quality of data in data warehouses
    - DW contains integrated, consistent, cleaned data
  - —Available information processing structure surrounding data warehouses
    - ODBC, OLEDB, Web accessing, service facilities, reporting and OLAP tools
  - —OLAP-based exploratory data analysis
    - Mining with drilling, dicing, pivoting, etc.
  - —On-line selection of data mining functions
    - Integration and swapping of multiple mining functions, algorithms, and tasks

从联机分析处理到联机分析挖掘

- 为什么要联机分析挖掘
  - —数据仓库中有高质量的数据
    - 数据仓库中存放着整合的、一致的、清理过的数据
  - —围绕数据仓库的信息处理结构
    - 存取、集成、合并多个异种数据库的转换，ODBC/OLEDB连接,Web访问和访问工具等
  - —基于OLAP的探测式数据分析
    - 使用上卷、下钻、切片、转轴等技术进行数据挖掘
  - —数据挖掘功能的联机选择
    - 多种数据挖掘功能、算法和任务的整合

联机分析挖掘的体系结构



Chapter 5-02: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- **Data Warehouse Implementation**
  数据仓库的实现
- Summary

数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

Efficient Data Cube Computation

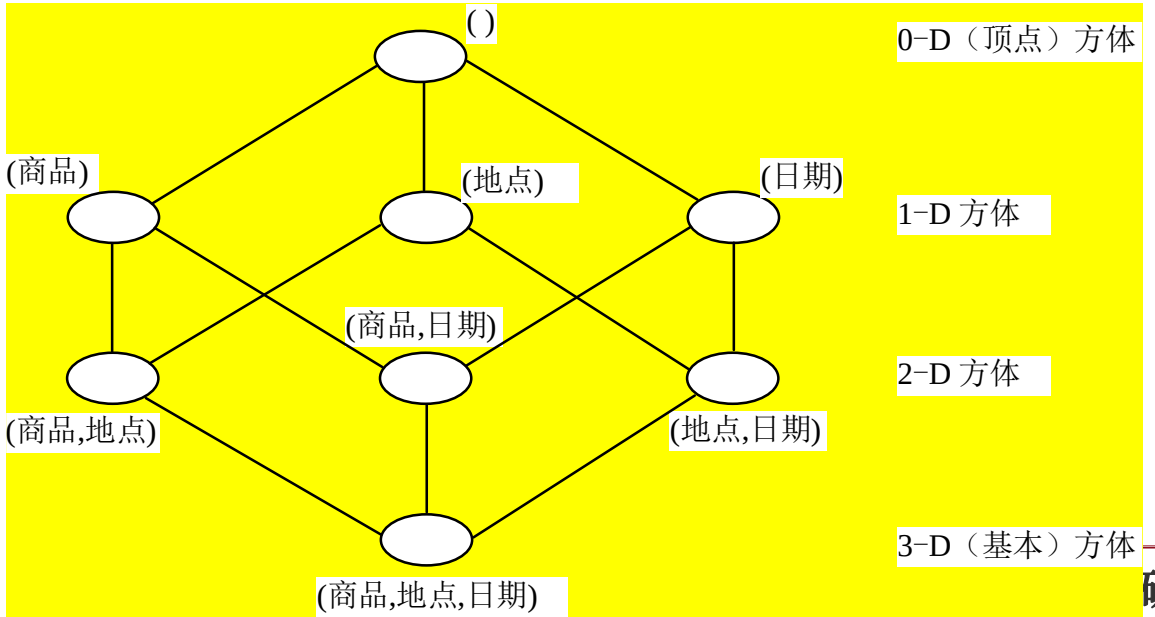
- Data cube can be viewed as a lattice of cuboids
  - —The bottom-most cuboid is the base cuboid
  - —The top-most cuboid (apex) contains only one cell
  - —How many cuboids in an n-dimensional cube with L levels?

$$T = \prod_{i=1}^{n} (L_i + 1)$$

- Materialization of data cube
  - —Materialize every (cuboid) (full materialization), none (no materialization), or some (partial materialization)
  - —Selection of which cuboids to materialize
    - Based on size, sharing, access frequency, etc.

数据立方体的预计算与维灾难

- 给定一个维的集合，例如，对于销售数据立方体(商品，地点，日期，销售量)，其维集合为{商品，地点，日期}，由它可以构造一个方体的格，其中0维方体存放最高层的汇总，称作**顶点方体**，是总销售量在所有的三个维汇总，也就是该立方体中所有数据的总销售量，顶点方体通常用All标记，它仅包含一个值。存放最底层汇总的方体称为**基本方体**，图中3维方体是给定维时间、地点和商品的基本方体。除基本方体外的其他方体称为**非基本方体**，基本方体中数据单元称为**基本单元**，非基本方体中数据单元称为**聚集单元**
- 每个方体在不同的汇总级显示数据，汇总级越高，其维数就越小，对应的数据量（元组个数）也越少，也就是说，一个k+1维方体的数据量大于其中在任一维上聚集得到的k维方体的数据量

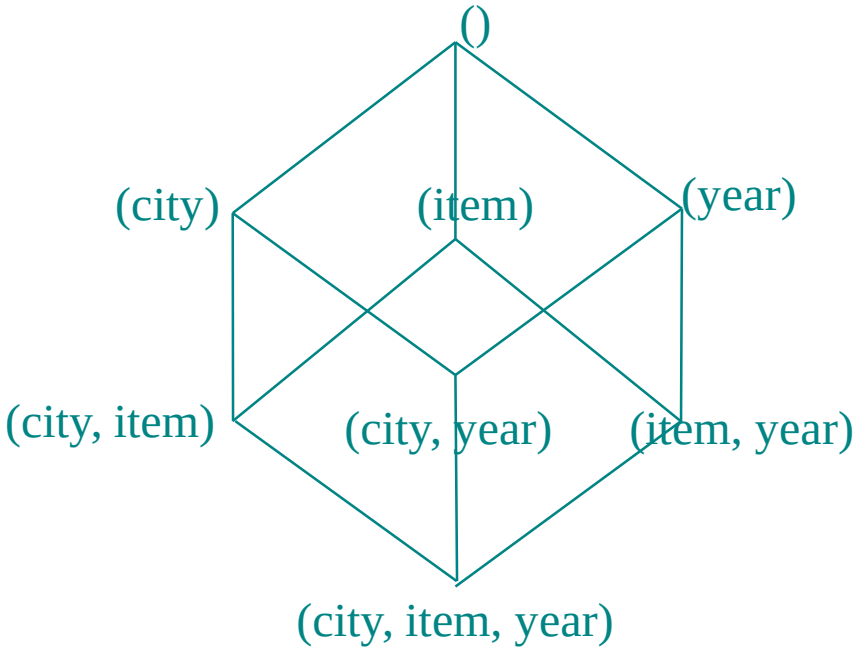


数据仓库的实现——数据立方体的有效计算

- 数据仓库中的OLAP查询是一种海量数据计算 （想象一下对过去10年各地区的软件产品销售的汇总查询）
  - —用户却希望这个计算能在数秒钟内完成
  - —解决方法在于给出一种有效的计算数据立方体的方法`
- 数据立方体可以被看成是一个***方体的格***
  - —最底层的方体是基本方体
  - —最顶端的方体（顶点）只包含一个单元的值
  - —一个n维的数据立方体，每维L层，可能产生的方体总数是多少？

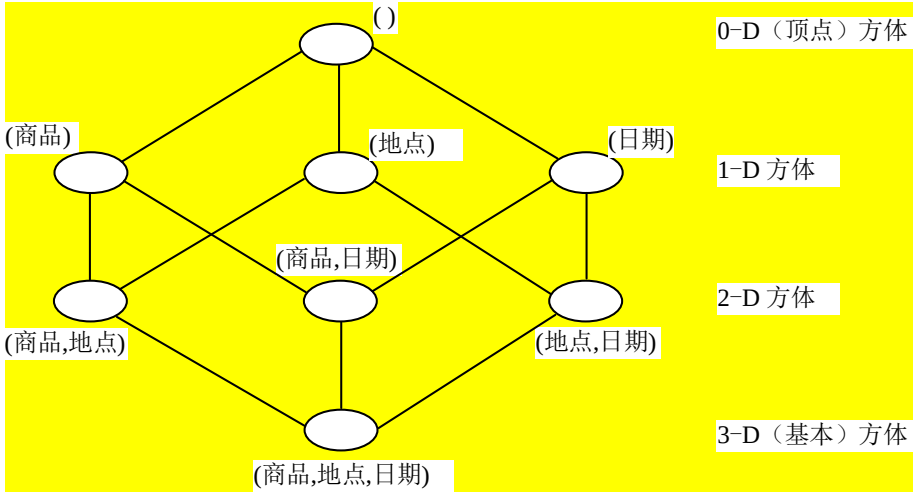
$$T = \prod_{i=1}^{n} (L_i + 1)$$

- **维灾难**
  - —**E.g. 假设数据立方体有10个维，每维5层，则可能产生的方体总数为 $5^{10} \approx 9.8 \times 10^6$个**



数据立方体的预计算与维灾难

- 一个OLAP查询问题可以转化成方体的计算
- 求某个*k*维方体有以下两种方法
  - （1）通过基本方体进行聚集产生*k*维方体
    - 由于基本方体维数最多，其数据量最大，所以该方法是十分耗时的
  - （2）从*k*+1维方体计算出*k*维方体
    - 如给定(商品，地点)这个2维方体，在各维上聚集可以产生(商品)和(地点)两个1维方体，显然比从(商品，地点，日期)基本方体产生(商品)和(地点)方体要省时，但需要预先计算出(商品，地点)方体



数据立方体的预计算与维灾难

- 从方法（2）看出，对于不同的查询分析，OLAP可能需要访问不同的方体
- 提前计算数据立方体中所有的或者一部分方体，这称为预计算。预计算带来快速的响应时间，并避免一些冗余计算
- 然而，预计算的主要挑战是，如果数据立方体中所有的方体都预先计算，所需的存储空间可能爆炸，特别是当立方体包含许多维，这许多维都具有多层的概念分层时，存储需求甚至更多，那么这个数据量究竟有多少呢？

  —给定n维{d1，d2，…，dn}的数据集，对于任何一个维di，假设在聚集时仅从自身或All两个值中选取，所以可以生成聚集方体的个数 $\prod_{i=1}^{n} 2 = 2^n$

数据立方体的预计算与维灾难

- 如果在聚集时，假设每个维d_i只有一层，聚集数据单元从维d_i（$1 \leqslant i \leqslant n$）的可能取值为该维的不同属性取值个数加上All，即$|d_i|+1$（$|d_i|$表示维d_i的不同取值个数），所以由n个维聚集的方体中聚集的数据单元个数为这n个维中各个维属性的可能取值个数的乘积，即 $\prod_{i=1}^{n}(|d_i|+1)$

- 如果在聚集时，假设每个维di有hi层，聚集数据单元从维di（1≤i≤n）的可能取值为该维的各层的属性取值，则可产生 $\prod_{i=1}^{n}(h_i+1)$ 个聚集方体

- 总的聚集数据单元个数为 $\prod_{i=1}^{n}\prod_{j=1}^{h_i}(|l_{i,j}|+1|)$ ，其中|li，j|表示维di的第j层上不同取值的个数

数据立方体的预计算与维灾难

- 如果一个多维数据集有10维，每维5层（含All层），则可能产生的方体总数是$5^{10}\approx9800000$，而总的聚集数据单元个数更是大的惊人
- 这种当数据立方体的维数较多和各维的层次较多时，可能的聚集计算量剧增，导致存储空间出现爆炸的现象称作维灾难

The "Compute Cube" Operator

- **Cube definition and computation in DMQL**

  define cube sales [item, city, year]: sum (sales_in_dollars)

  compute cube sales

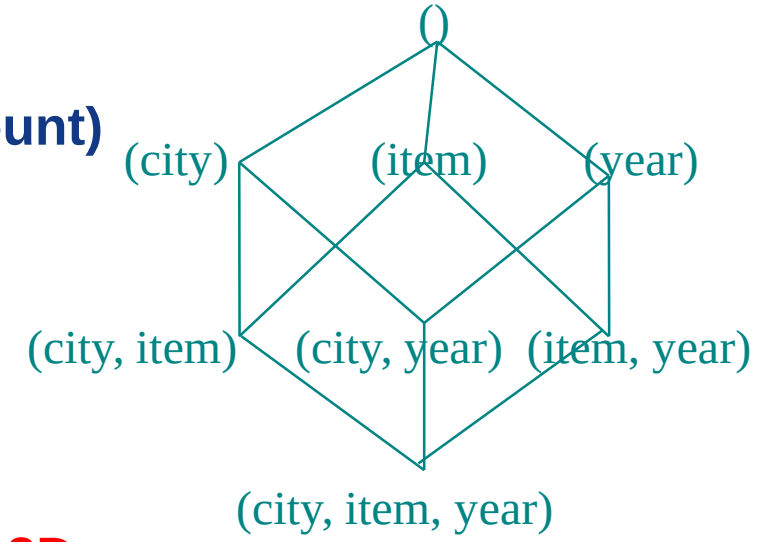
- **Transform it into a SQL-like language (with a new operator cube by, introduced by Gray et al.'96)**

  SELECT item, city, year, SUM (amount)

  FROM SALES

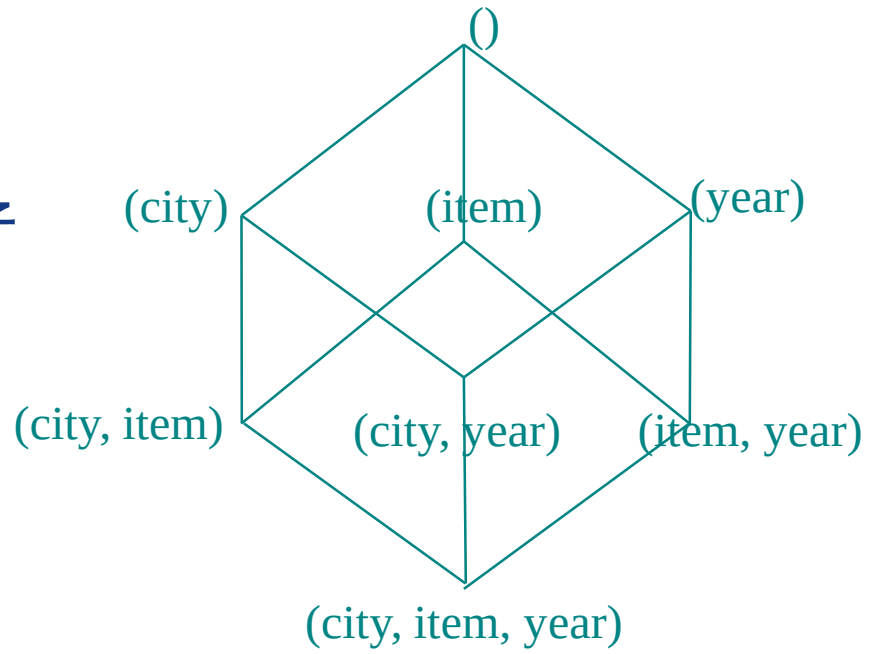
  CUBE BY item, city, year

- **Need compute the following Group-Bys**
  - **(item, city, year) –3D**
  - **(item, city), (item year), (city, year) —2D**
  - **(item), (city), (year) —1D**
  - **() --0D**



数据仓库的实现—— 方体的操作

- DMQL中的方体定义和计算
  - — *define cube* sales[item, city, year]: sum(sales_in_dollars)
  - — *compute cube* sales
- 上述的compute cube子句可以转化为一个类似于SQL的语句
  - — SELECT item, city, year, SUM (amount)
  - — FROM SALES
  - — *CUBE BY* item, city, year
- 这个相当于SQL中以下的group by子句
  - — (item, city, year) –3D
  - — (item, city), (item year), (city, year) —2D
  - — (item), (city), (year) —1D
  - — () --0D



一种数据挖掘查询语言: DMQL

- DMQL首先包括定义数据仓库和数据集市的语言原语，这包括两种原语定义：一种是立方体定义，一种是维定义
  - —立方体定义 (事实表)
    - define cube <cube_name> [<dimension_list>]: <measure_list>
  - —维定义 (维表)
    - define dimension <dimension_name> as (<attribute_or_subdimension_list>)
  - —特殊案例 (共享维表的定义)
    - 第一次作为维表定义 “cube definition”
    - 然后：define dimension <dimension_name> as <dimension_name_first_time> in cube <cube_name_first_time>

实例：使用DMQL定义星型模式

```
define cube sales_star [time, item, branch,
    location]:
        dollars_sold = sum(sales_in_dollars),
          avg_sales = avg(sales_in_dollars), units_sold
          = count(*)
define dimension time as (time_key, day,
    day_of_week, month, quarter, year)
define dimension item as (item_key, item_name,
    brand, type, supplier_type)
define dimension branch as (branch_key,
    branch_name, branch_type)
define dimension location as (location_key, street,
    city, province_or_state, country)
```

实例：使用DMQL定义雪花模式

define cube sales_snowflake [time, item, branch, location]:

dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)

define dimension time as (time_key, day, day_of_week, month, quarter, year)

define dimension item as (item_key, item_name, brand, type, supplier(supplier_key, supplier_type))

define dimension branch as (branch_key, branch_name, branch_type)

define dimension location as (location_key, street, city(city_key, province_or_state, country))

实例：使用DMQL定义事实星座模式

```
define cube sales [time, item, branch, location]:
        dollars_sold = sum(sales_in_dollars), avg_sales =
          avg(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter,
    year)
define dimension item as (item_key, item_name, brand, type,
    supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city,
    province_or_state, country)
define cube shipping [time, item, shipper, from_location, to_location]:
        dollar_cost = sum(cost_in_dollars), unit_shipped = count(*)
define dimension time as time in cube sales
define dimension item as item in cube sales
define dimension shipper as (shipper_key, shipper_name, location as
    location in cube sales, shipper_type)
define dimension from_location as location in cube sales
define dimension to_location as location in cube sales
```

数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

数据立方体的压缩存储

- 通常数据立方体中包含海量数据，为了节省存储空间，人们提出了各种数据压缩方法
- 对于给定的数据立方体，由于度量的不同值个数可能很大，难以压缩存储，主要是对维进行压缩存储

维 1	维 2	维 3	维 4
DMP1	40	CMOS	Yes
DMP2	20	CMOS	Yes
DMP3	20	TTL	Yes
DMP4	20	TTL	Yes
DMP5	28	CMOS	No
DMP6	40	CMOS	No
DMP7	28	CMOS	No

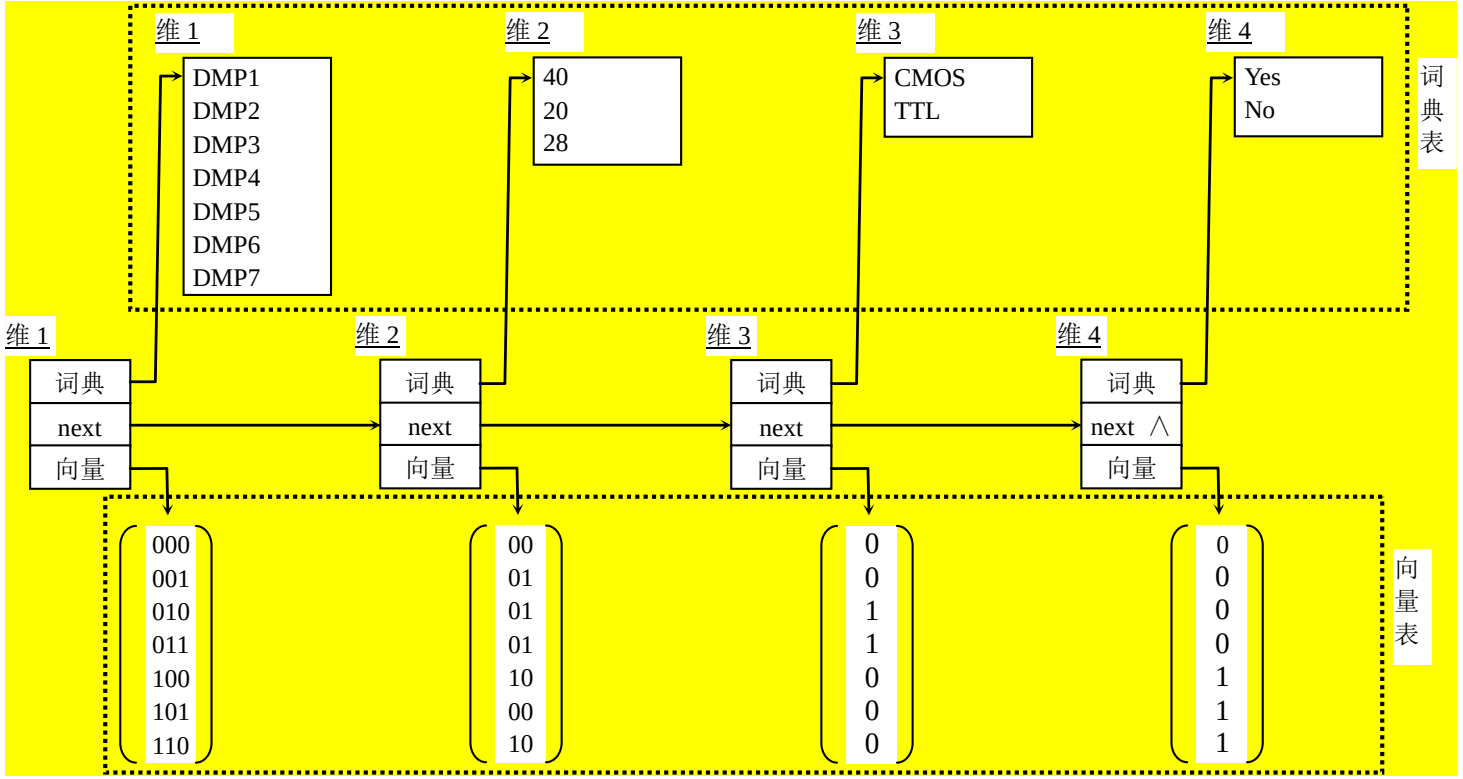
压缩存储

维 1	维 2	维 3	维 4
000	00	0	0
001	01	0	0
010	01	1	0
011	01	1	0
100	10	0	1
101	00	0	1
110	10	0	1

对于数据立方体的一个维度d_i，求出其中不同的维成员个数为$|d_i|$，采用二进制编码，对应的二进制位数为m_i，它为满足条件$\geq |d_i|$的最小m_i，对于左图所示的数据立方体，m_1=3，m_2=2，m_3=1，m_4=1，编码后的数据立方体如右图所示

数据立方体的压缩存储

- 为方便存取，设计对应的压缩存储结构如图，词典表中包含各维成员的编码表
  - —维2的编码表中，40对应编码为00，其物理地址也为00，这设计便于高效查找
  - —向量表中包含各维对应的事实数据，以编码方式存放，通过相同地址关联，如维1向量的第2行、维2向量的第2行、维3向量的第2行、维4向量的第2行构成第2个事实元组



数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

Indexing OLAP Data: Bitmap Index

- Index on a particular column在特定栏上的索引
- Each value in the column has a bit vector: bit-op is fast这一栏上的每一个值都对应于一个位向量
- The length of the bit vector: # of records in the base table位向量的长度：基本表中特定栏属性值的个数
- The i-th bit is set if the i-th row of the base table has the value for the indexed column如果基本表中的给定行的属性值为v，则在为图索引的对应行，表示改值的位为1，该行的其它位均为0
- not suitable for high cardinality domains对于基数较大的域不大适合
  - A recent bit compression technique, Word-Aligned Hybrid (WAH), makes it work for high cardinality domain as well [Wu, et al. TODS'06]

Base table

Cust	Region	Type
C1	Asia	Retail
C2	Europe	Dealer
C3	Asia	Dealer
C4	America	Retail
C5	Europe	Dealer

Index on Region

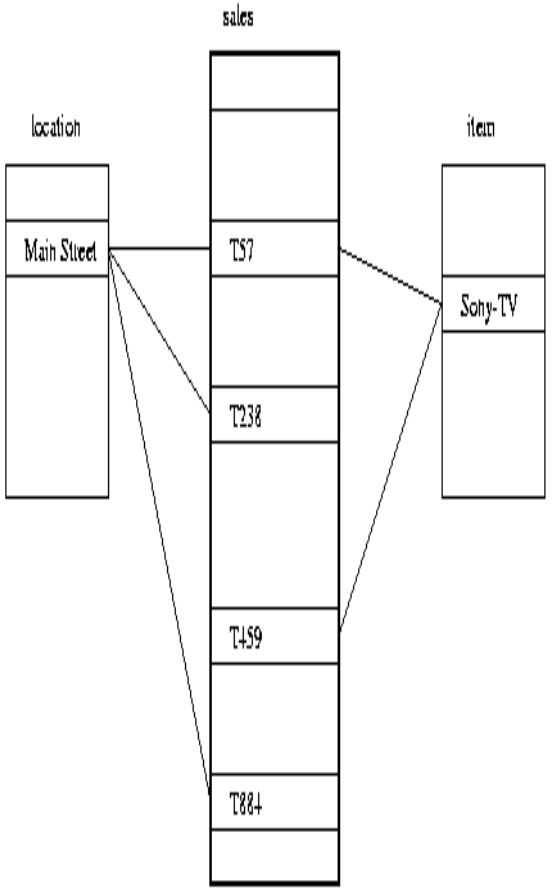
RecID	Asia	Europe	America
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

Index on Type

RecID	Retail	Dealer
1	1	0
2	0	1
3	0	1
4	1	0
5	0	1

Indexing OLAP Data: Join Indices连接索引

- Join index: JI(R-id, S-id) where R (R-id, …) ⋈ S (S-id, …)
- 连接索引: 如果两个关系R(RID ,A)和S(B,SID)在属性A和B上连接，则连接索引记录包含JI（RID,SID)对，其中RID和SID分别来自R和S的记录标识符
- Traditional indices map the values to a list of record ids
  - It materializes relational join in JI file and speeds up relational join
- 传统的索引将给定列上的值映射到具有该值的列表上
  - 它物化了JI文件中的关系连接并加速了关系连接——这是一个相当昂贵的操作
- In data warehouses, join index relates the values of the dimensions of a start schema to rows in the fact table.
  - E.g. fact table: *Sales* and two dimensions *city* and *product*
    - A join index on *city* maintains for each distinct city a list of R-IDs of the tuples recording the Sales in the city
  - Join indices can span multiple dimensions
- 在数据仓库中，连接索引把星型模式的维值连接到事实表的行
  - 例如：对sales 事实表和city,product 两个维,城市的连接索引使每一个不同的城市有一个元组的记录标识符列表记录个城市的销售
  - 连接索引可以跨越多维



Efficient Processing OLAP Queries

- Determine which operations should be performed on the available cuboids
  - Transform drill, roll, etc. into corresponding SQL and/or OLAP operations, e.g., dice = selection + projection
- Determine which materialized cuboid(s) should be selected for OLAP op.
  - Let the query to be processed be on {*brand, province_or_state*} with the condition "*year = 2004*", and there are 4 materialized cuboids available:
    1) {*year, item_name, city*}
    2) {*year, brand, country*}
    3) {*year, brand, province_or_state*}
    4) {*item_name, province_or_state*} where *year = 2004*

    Which should be selected to process the query?
- Explore indexing structures and compressed vs. dense array structs in MOLAP

OLAP Server Architectures

- Relational OLAP (ROLAP)
  - Use relational or extended-relational DBMS to store and manage warehouse data and OLAP middle ware
  - Include optimization of DBMS backend, implementation of aggregation navigation logic, and additional tools and services
  - Greater scalability
- Multidimensional OLAP (MOLAP)
  - Sparse array-based multidimensional storage engine
  - Fast indexing to pre-computed summarized data
- Hybrid OLAP (HOLAP) (e.g., Microsoft SQLServer)
  - Flexibility, e.g., low level: relational, high-level: array
- Specialized SQL servers (e.g., Redbricks)
  - Specialized support for SQL queries over star/snowflake schemas

Chapter 5-02: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- **Summary**

Summary

- Data warehousing: A multi-dimensional model of a data warehouse
  - A data cube consists of *dimensions* & *measures*
  - Star schema, snowflake schema, fact constellations
  - OLAP operations: drilling, rolling, slicing, dicing and pivoting
- Data Warehouse Architecture, Design, and Usage
  - Multi-tiered architecture
  - Business analysis design framework
  - Information processing, analytical processing, data mining, OLAM (Online Analytical Mining)
- Implementation: Efficient computation of data cubes
  - Partial vs. full vs. no materialization
  - Indexing OALP data: Bitmap index and join index
  - OLAP query processing
  - OLAP servers: ROLAP, MOLAP, HOLAP

数据仓库以及 OLAP 实验

- 数据仓库构建实验
  - 运用ETL工具对数据库sakila中的数据进行清洗操作，构建一个DVD租赁商店数据仓库系统，实现定期从源数据库sakila中抽取增量数据，转换成符合DVD租赁业务的数据，最后加载到目标数据仓库中
- OLAP 实验
  - 进行的 OLAP 操作（上卷、下钻、旋转、切块、切片等）